File Operation Failure Diagnoser

BCSE303P – Operating Systems Lab

MINI-PROJECT COMPONENT REPORT

by

Yash Pradhan

(24BCE0702)

Anjini Pandey

(24BCE0714)

Dhruv Kothari

(24BCE0570)

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

OCTOBER 2026

DECLARATION

We hereby declare that the report entitled “File Operation Failure Diagnoser” submitted by Yash Pradhan (24BCE0702), Anjini Pandey (24BCE0714) and Dhruv Kothari (24BCE0570), for the BCSE303P Operating Systems Lab to VIT is a record of bonafide work carried out by us under the supervision of Dr. N. Narayanan Prasanth.

We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for any other courses in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates

Yash Pradhan (24BCE0702)

Anjini Pandey (24BCE0714)

Dhruv Kothari (24BCE0570)

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. N. Narayanan Prasanth, under whose supervision this mini-project was carried out for the BCSE303P Operating Systems Lab, for his guidance throughout the project and for the opportunity to present our work at each review stage.

We thank the School of Computer Science and Engineering, Vellore Institute of Technology, for the academic environment and the laboratory course that made this work possible.

We also acknowledge the authors and maintainers of the official Microsoft, Python, psutil and pytest documentation, whose material underpins the technical design described in this report, and the textbooks listed in the references that taught us the operating-system concepts applied here.

Finally, we thank each other for the collaboration, discussion and shared effort that carried the project from the problem statement in Review 1 to the verified implementation reported here.

Yash Pradhan
Anjini Pandey
Dhruv Kothari

ABSTRACT

Operating systems protect shared files from corrupting concurrent access, but the messages shown when that protection triggers — “Access is denied”, “The file is in use” — rarely identify the cause. The File Operation Failure Diagnoser is a Windows desktop application that explains why a selected file operation (READ, WRITE, EXECUTE or DELETE) would fail; it diagnoses the operation and never performs it on the target.

A Tkinter GUI passes the path and operation to a Diagnosis Engine that queries four independent peer analyzers: Process / Lock (Windows Restart Manager through `ctypes`), Permission / Ownership (`os.access()` and Win32 owner resolution), File Attribute (read-only attribute inspection) and Cloud-Sync (local OneDrive and Google Drive evidence). The engine correlates the structured evidence with a deterministic priority policy that selects one primary diagnosis while preserving secondary findings, and a Recommendation Engine produces advisory, evidence-grounded guidance.

The final regression record (Windows 11 Pro 64-bit, Python 3.13.14, pytest 9.1.1) shows 157 tests collected, 156 passed, 0 failed and 1 skipped (a platform-specific test). A live GUI demonstration covered fourteen scenario groups in sixteen executions: fifteen matched expectation and one permission-restriction execution is reported as BLOCKED, because the Restart Manager query returned Win32 error 5 and the application reported UNKNOWN_ERROR instead of PERMISSION_DENIED. The cloud-sync screenshot is a negative boundary case; confirmed provider-lock correlation is verified by automated tests only.

Keywords: file-operation failure, Windows Restart Manager, file locks, NTFS ownership, file attributes, cloud synchronization, diagnosis engine, Tkinter.

TABLE OF CONTENTS

Page No.

Acknowledgement ..... **i**

Abstract ..... **ii**

Table of Contents ..... **iii**

List of Figures ..... **v**

List of Tables ..... **vi**

List of Abbreviations ..... **vii**

Symbols and Notations ..... **viii**

1. INTRODUCTION **1**

1.1 Objectives 1
1.2 Motivation 1
1.3 Background 2
1.4 Proposed solution 2
1.5 Scope and boundaries 2
1.6 Evolution since Review 1 and Review 2 2
1.7 Summary of the project at a glance 3

2. LITERATURE SURVEY **4**

2.1 Problem Statement 4
2.2 Architectural foundations 4
2.3 Existing diagnostic and unlocking utilities 4
2.4 Comparison 5
2.5 Gap addressed 5

3. TECHNICAL SPECIFICATION **6**

3.1 Hardware and software environment 6
3.2 Interfaces used by the implementation 6
3.3 Windows Restart Manager 7
3.4 Permission and ownership evidence 8
3.5 File attribute semantics 8
3.6 Cloud-sync discovery sources 9
3.7 Platform handling 9
3.8 Source-code organisation 9

4. DESIGN **10**

4.1 Development methodology 10
4.2 System architecture 10
4.3 Design principles 12
4.4 Data model 12
4.5 End-to-end workflow 13
4.6 Diagnosis-priority rules 15
4.7 Why a deterministic policy, and why secondary evidence is preserved 18
4.8 Worked traces 18
4.9 GUI design 19

5. PROPOSED SYSTEM.....20
5.1 Process / Lock Analyzer.....20
5.2 Permission / Ownership Analyzer.....21
5.3 File Attribute Analyzer.....23
5.4 Cloud-Sync Analyzer.....24
5.5 Diagnosis Engine and evidence correlation.....26
5.6 Recommendation Engine.....26
5.7 Tkinter GUI.....28
6. RESULTS AND DISCUSSION.....29
6.1 Evidence capture method.....29
6.2 Demonstration screenshots.....30
6.3 Testing methodology.....39
6.4 Final regression result.....40
6.5 Historical milestone totals (kept separate).....41
6.6 The skipped test.....41
6.7 Live demonstration counts.....41
6.8 Scenario matrix.....42
6.9 What each kind of evidence establishes.....42
6.10 Key findings.....43
6.11 Limitations.....43
6.12 Observed GUI limitations.....44
6.13 Discussion.....44
7. CONCLUSION.....45
7.1 Conclusion.....45
7.2 Future improvements.....45
REFERENCES.....46
APPENDIX A. ADDITIONAL SOURCE EXCERPTS.....48
A.1 Failure isolation in the Diagnosis Engine.....48
A.2 Analyzer base class.....48
A.3 Sync-root membership and client detection.....49
A.4 Recommendation for a confirmed sync conflict.....49
A.5 GUI details.....49
APPENDIX B. EVIDENCE INDEX.....51

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Restart Manager call sequence in ProcessLockAnalyzer._detect_locks_restart_manager	7
4.1	Final system architecture	11
4.2	Diagnosis pipeline from user input to the displayed result	14
4.3	Order of evaluation in _classify()	16
6.1	Scenario 03, top viewport	31
6.2	Scenario 03, bottom viewport of the same diagnosis	31
6.3	Scenario 08: WRITE on a file that is both held open and read-only	32
6.4	Scenario 04a: WRITE on 02_read_only_file.txt → READ ONLY	33
6.5	Scenario 04b: the same file with READ → OPERATION ALLOWED	33
6.6	Scenario 01b: WRITE on 01_normal_file.txt → OPERATION ALLOWED	34
6.7	Scenario 02: READ on a path that does not exist → FILE NOT FOUND	34
6.8	Scenario 06: READ on 03_hidden_archive_file.txt → OPERATION ALLOWED	35
6.9	Scenario 13: READ on 04_system_attribute_file.txt → OPERATION ALLOWED	35
6.10	Scenario 10: EXECUTE on 08_execute_check.cmd → OPERATION ALLOWED	36
6.11	Scenario 11: DELETE on 07_delete_check_file.txt → OPERATION ALLOWED	36
6.12	Scenario 12: DELETE on the read-only fixture → READ ONLY	37
6.13	Scenario 14: a normal file diagnosed after earlier runs in the same window	37
6.14	Scenario 09: a directory selected as the target → UNKNOWN ERROR, “The specified path is a directory, not …	38
6.15	Scenario 07: cloud-sync boundary behaviour for 01_normal_file.txt, which lies outside any detected …	38
6.16	Scenario 05 (BLOCKED): READ on 06_permission_restricted_file.txt, which carries an explicit deny ACE	39

LIST OF TABLES

Table No.	Title	Page No.
1.1	Review 1 objectives mapped to the final implementation	1
1.2	What the final application does and does not do	2
1.3	Key facts at a glance	3
2.1	Capability comparison (existing tools as summarised in the Review 1 and Review 2 surveys)	5
3.1	Software environment of the final verification run	6
3.2	Operating-system interfaces and libraries used	6
3.3	Win32 attribute flags decoded by FileAttributeAnalyzer	8
3.4	How supported provider roots and clients are located	9
3.5	Application source files	9
4.1	Review 3 development milestones (from the engineering log)	10
4.2	Architectural elements and their implementing code	11
4.3	Structured result types	13
4.4	Rule rationale	17
4.5	How selected scenarios resolve through the pipeline	18
4.6	GUI panels and the result data they display	19
5.1	Cloud-sync evidence tiers	25
5.2	Advisory guidance by diagnosis (summarised from recommender.py)	27
6.1	Guide to the demonstration figures	30
6.2	Evidence observed in Scenario 08 (Figure 6.3)	32
6.3	Verification layers	40
6.4	Final verified regression record	40
6.5	Final-run test counts by file (from the regression log)	40
6.6	Historical totals as recorded at each milestone	41
6.7	Live GUI demonstration counts	41
6.8	Live GUI scenario results (Review 3 evidence manifest, v5)	42
6.9	Live demonstration versus automated evidence	42
6.10	Documented limitations of the final implementation	43
7.1	Future work (not implemented)	45
B.1	Screenshot files by scenario	51

LIST OF ABBREVIATIONS

ACE	Access Control Entry
API	Application Programming Interface
DACL	Discretionary Access Control List
DLL	Dynamic-Link Library
DriveFS	File-system name of Google Drive for desktop volumes, as recognised by the Cloud-Sync Analyzer
DWORD	Double word (32-bit unsigned integer)
GDI+	Graphics Device Interface Plus (used for window capture)
GUI	Graphical User Interface
IEEE	Institute of Electrical and Electronics Engineers
NTFS	New Technology File System
OS	Operating System
PID	Process Identifier
PNG	Portable Network Graphics
RM	Restart Manager
SID	Security Identifier
VIT	Vellore Institute of Technology
Win32	32-bit Windows application programming interface

SYMBOLS AND NOTATIONS

0xNNNN	Hexadecimal value, e.g. 0x0020 is the archive attribute bit; 0x0021 is read-only plus archive
[n]	IEEE numbered citation of reference n
⋮	Lines omitted between two ranges of a source-code excerpt
→	“Leads to” or “maps to” in diagrams and tables
Rule n	The n-th condition tested in DiagnosisEngine._classify()
Scenario 01a–14	Scenario identifiers of the Review 3 evidence manifest (v5)
Error 5 / 234	Win32 error codes ERROR_ACCESS_DENIED and ERROR_MORE_DATA
code font	Identifiers, file names and API names taken from the source or the Win32 documentation

1. INTRODUCTION

1.1 Objectives

The primary objective of this project is to create an intuitive desktop application that actively diagnoses and explains file operation failures. Table 1.1 maps each objective set in Review 1 to its final realisation and the evidence that supports it.

Table 1.1. Review 1 objectives mapped to the final implementation

Objective (Review 1)	Final realisation	Evidence
Process and handle detection	`ProcessLockAnalyzer` uses Windows Restart Manager to return process name, PID and path (enriched with `psutil`)	Scenario 03 live capture; Section 5.1
File-system attribute analysis	`FileAttributeAnalyzer` decodes read-only, hidden, system, archive, reparse-point and offline flags and shows the raw bitmask	Scenarios 04a, 04b, 06, 12, 13
Permission and ownership analysis	`PermissionAnalyzer` combines `os.access()` results with Win32 owner resolution; the GUI states that full NTFS DACL evaluation is not performed	Scenarios 01–14 permission panel; Section 6.11
Synchronization conflict identification	`SyncConflictAnalyzer` supports OneDrive and Google Drive only, using four evidence tiers; a confirmed conflict requires a lock held by the provider's client process	Scenario 07 (negative boundary); automated tests
Actionable, safe recommendations	`Recommender` produces advisory text per diagnosis; no action is executed by the tool	Recommendation panel in every screenshot
User-interface development	Tkinter desktop GUI with path entry, Browse dialog, operation selector and six result panels	Section 6.2

1.2 Motivation

Review 1 sets out to bridge the gap between operating-system resource management — process handles and file-system locks in particular — and end-user troubleshooting. The OS correctly enforces protection and mutual exclusion, yet its abstraction withholds what a user needs to resolve a conflict; a diagnostic tool can turn that hidden state into an explanation.

- **Bridge OS and user:** translate low-level resource state into actionable guidance.
- **Apply core OS concepts:** process management, system interfaces, file systems, protection and synchronization in one practical tool.
- **Speed up troubleshooting:** avoid unnecessary restarts and unsafe interventions such as force-killing processes.
- **Educational value:** show observable OS behaviour, such as why a read-only file can be read but not written.

1.3 Background

A multitasking operating system must let many programs share files without corrupting them. It does so with file handles, sharing modes, access-control information and file attributes, all enforced inside the kernel and file-system layers [1, 2]. Windows manages these resources through its object and handle subsystems [3]. The enforcement is correct, but the abstraction hides the reason from the user: when two programs conflict, or when a permission or attribute blocks an action, the visible result is a short generic message.

1.4 Proposed solution

The File Operation Failure Diagnoser accepts a file path and an intended operation, gathers operating-system evidence from four independent analyzers, correlates that evidence, and reports a single primary diagnosis with a plain-language reason and a safe recommendation. Secondary findings are retained and shown beside the primary diagnosis so that the user sees the whole picture without the headline overstating the cause.

1.5 Scope and boundaries

Table 1.2. What the final application does and does not do

The application does	The application does not
Diagnose a selected operation (READ, WRITE, EXECUTE, DELETE) on a selected file	Perform the selected operation: it never reads, writes, executes or deletes the target
Report holding processes (name, PID, executable path where available) through Windows Restart Manager	Terminate, suspend or inject into any process, or close handles
Report preliminary access results from `os.access()` and the file owner resolved through Win32 security APIs	Evaluate complete NTFS DACL inheritance or parent-directory delete rights
Decode six Win32 attribute flags (read-only, hidden, system, archive, reparse point, offline)	Change attributes, ownership, permissions or registry values
Detect local OneDrive and Google Drive association and client activity, with conservative evidence tiers	Contact cloud services, support other providers, or treat a running client or a path inside a sync root as proof of a conflict
Present advisory, evidence-grounded recommendations	Apply any recommendation automatically

The primary lock-detection mechanism and the Win32 owner lookup are Windows-specific. The code base degrades gracefully on other platforms for development and testing, but Windows 10/11 is the supported target.

1.6 Evolution since Review 1 and Review 2

The final scope differs from the earliest plan in three ways:

- **Advisory only.** Review 1 mentioned optional action buttons such as “Terminate Process”. Review 2 adopted a safe-recommendation policy, and the final source contains no process-termination code. Recommendations are text.
- **File selection.** Review 1 envisaged a context-menu or drag-and-drop interface. The final GUI selects files through a path entry box and a Browse dialog; neither shell-context-menu integration nor drag-and-drop is implemented.
- **Analyzers and permissions.** At Review 2 the engine had a three-step flow and the attribute and sync analyzers were proposed; all four now feed a seven-rule policy. Early ACL-comparison wording became `os.access()` access plus the Win32 owner, with the full NTFS DACL explicitly not evaluated.

1.7 Summary of the project at a glance

Table 1.3. Key facts at a glance

Item	Final verified value
Application	Windows desktop diagnostic tool; Python, Tkinter GUI; package `diagnoser` (about 3,700 lines including docstrings)
Peer analyzers	Process / Lock · Permission / Ownership · File Attribute · Cloud-Sync (OneDrive and Google Drive only)
Diagnosis categories	`FILE_NOT_FOUND`, `FILE_LOCKED`, `PERMISSION_DENIED`, `READ_ONLY`, `SYNC_CONFLICT`, `OPERATION_ALLOWED`, `UNKNOWN_ERROR`
Automated regression (final record)	157 collected · 156 passed · 0 failed · 1 skipped (Windows 11 Pro 64-bit, Python 3.13.14, pytest 9.1.1)
Live GUI demonstration	14 logical groups / 16 executions · 15 executions (13 groups) matched expectation · 1 execution BLOCKED (permission restriction)
Safety policy	Read-only and advisory: no process termination, no file, attribute, ACL or registry modification

2. LITERATURE SURVEY

2.1 Problem Statement

When users delete, move, rename, copy or save a file, Windows may refuse with messages such as “File is in use by another program”, “Access Denied” or “Action cannot be completed”. These messages state that the operation failed but not which process, permission, attribute or synchronization client is responsible. Review 1 identified three consequences: users do not know which application is blocking the file, they spend time on trial-and-error fixes (often a full restart), and they risk losing unsaved work or taking unsafe actions.

Typical situations named in the Review 1 presentation include saving a document in OneDrive while another application is accessing it, deleting a folder that is still open in File Explorer or being scanned by antivirus software, and moving a file from an external drive during a temporary read/write conflict.

2.2 Architectural foundations

Standard operating-system texts describe process management and file-system protection as core kernel responsibilities, with handles, sharing rules and permissions used to control concurrent access and preserve integrity [1, 2]. Windows Internals gives the Windows-specific background on processes, objects and handles [3]. Cloud-sync clients such as OneDrive and Google Drive for desktop keep local files synchronised with the cloud and can present files as online-only or locally available [4, 5]; because they run in the background, they are one more program that users may not think of when a file is reported as in use.

2.3 Existing diagnostic and unlocking utilities

Sysinternals Handle and Process Explorer. Handle is a command-line utility that displays information about open handles for any process, so it can show which programs have a file open; it must be run with administrative privilege, and its documentation warns that closing handles can cause application or system instability. Process Explorer is its GUI counterpart [6]. Handle enumerates open handles directly, whereas the Diagnoser relies on Restart Manager (Section 3.3). These Sysinternals tools are aimed at developers and administrators, and their documentation does not describe combining permission, attribute or synchronization evidence into a user-oriented diagnosis.

PowerToys File Locksmith. File Locksmith is a shell extension that scans the running processes it can access and lists those using selected files or directories; processes run by another user

may be missing unless it is restarted as administrator, and the list offers an “End task” action [7]. It is convenient for process-based locks but is process-centred by design.

LockHunter. LockHunter shows the processes locking a file or folder and can unlock, delete, copy or rename the file, delete it at the next restart, or kill the locking process [8]. Its philosophy is to release the file, not to explain why access failed, and forcing a process to exit without understanding it can interrupt unsaved work.

2.4 Comparison

Table 2.1. Capability comparison (existing tools as summarised in the Review 1 and Review 2 surveys)

Capability	Process Explorer / Handle	File Locksmith	LockHunter	This project (final)
Process / lock detection	Yes	Yes	Yes	Yes — Restart Manager
Permission / ownership evidence	Partial	No	No	Yes — `os.access()` + owner (no full DACL)
File-attribute evidence	No	No	Partial	Yes — six decoded flags + raw bitmask
Cloud-sync evidence	No	No	No	Yes, limited — OneDrive and Google Drive, local evidence only
Diagnosis style	Technical	Technical	Action-oriented	Plain-language diagnosis with advisory guidance
Acts on processes	Can close handles	Can end tasks	Can kill / unlock / delete	Never — diagnostic only

Lock-detection and action entries reflect the tools’ official documentation [6, 7, 8]; the permission, attribute and sync entries summarise the team’s Review 1 and Review 2 survey and are not a systematic benchmark.

2.5 Gap addressed

The surveyed tools are developer-oriented handle viewers, process-only checkers, and utilities that act on the locking process. The documentation of the tools surveyed does not describe combining lock, permission, attribute and synchronization evidence into one explanation; the Diagnoser targets that combination, within the limits stated in Section 6.11. Its differentiator is correlating several operating-system evidence sources, ranking them deterministically and keeping secondary evidence visible.

3. TECHNICAL SPECIFICATION

3.1 Hardware and software environment

The application is a user-mode Python program for Windows. The environment in which the final regression suite and the live demonstration were run is given in the table below. No hardware specification beyond the single Windows 11 machine is claimed.

Table 3.1. Software environment of the final verification run

Component	Specification
Operating system	Windows 11 Pro 64-bit (supported target: Windows 10 / 11)
Language and runtime	Python 3.13.14 (the source states Python 3.8 or later as a minimum)
GUI toolkit	Tkinter 8.6 with `ttk`
Third-party library	`psutil` (declared as `psutil>=5.9.0`) for process metadata, local sync-client discovery and drive mounts
Standard-library interfaces	`os`, `ctypes`, `winreg`, `getpass`, `dataclasses`, `enum`
Test framework	pytest 9.1.1 (declared as `pytest>=7.0.0`)
Privileges	The application only reads state and modifies nothing; the evidence pack does not record the privilege level of the test account, so none is claimed

3.2 Interfaces used by the implementation

The table lists only interfaces that the supplied source actually calls. Windows interfaces are reached through `ctypes` foreign-function calls; everything else is Python standard library or `psutil`.

Table 3.2. Operating-system interfaces and libraries used

Interface	Module / DLL	Used in	Purpose
`RmStartSession`, `RmRegisterResources`, `RmGetList`, `RmEndSession`	`rstrtmgr.dll` via `ctypes`	`process_lock.py`	Primary lock detection: list the applications and services Restart Manager reports for the registered file [9, 10, 11, 12]
`GetNamedSecurityInfoW`	`advapi32.dll` via `ctypes`	`permissions.py`	Retrieve the owner SID of the file [13]
`LookupAccountSidW`	`advapi32.dll` via `ctypes`	`permissions.py`	Resolve the owner SID to `DOMAIN\User` [14]
`LocalFree`	`kernel32.dll` via `ctypes`	`permissions.py`	Release the security descriptor returned by `GetNamedSecurityInfoW`
`os.access`	Python `os` [15]	`permissions.py`, `process_lock.py`	Preliminary READ / WRITE / EXECUTE access checks; DELETE approximated by write
`os.stat().st_file_attributes`	Python `os` [16]	`file_attributes.py`, `sync_conflict.py`	Primary source of the Win32 attribute bitmask

Interface	Module / DLL	Used in	Purpose
`GetFileAttributesW`	`kernel32.dll` via `ctypes`	`file_attributes.py`	Fallback attribute retrieval [17]
`GetVolumeInformationW`	`kernel32.dll` via `ctypes`	`sync_conflict.py`	Recognise a Google Drive (DriveFS) volume by label or file-system name [18]
`winreg`	Python standard library [19]	`sync_conflict.py`	Read OneDrive and Google Drive folder locations from the registry
`psutil.Process.exe`, `psutil.process_iter`, `psutil.disk_partitions`	`psutil` [20]	`process_lock.py`, `sync_conflict.py`	Executable path of a holder; local sync-client discovery; drive-mount discovery
`ctypes`	Python standard library [21]	Analyzers	Foreign-function interface to the DLLs above
`tkinter`, `tkinter.ttk`	Python standard library [22]	`gui/`	Desktop user interface
`pytest`	Third-party test framework [23]	`tests/`	Automated regression suite

3.3 Windows Restart Manager

Restart Manager lets an installer find the applications and services that are using files it needs to replace [24]. A session is started and ended with `RmStartSession` and `RmEndSession`, files are registered with `RmRegisterResources`, and `RmGetList` returns the applications and services currently using the registered resources [9, 10, 11, 12]. The Diagnoser uses it read-only: it registers one file, reads the list and closes the session. It never calls the shutdown or restart functions listed in Microsoft's overview [25]. Figure 3.1 shows the call sequence.

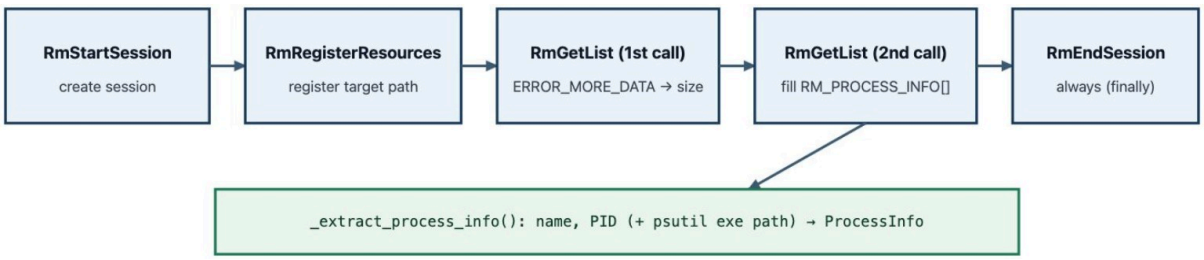


Figure 3.1. Restart Manager call sequence in `ProcessLockAnalyzer._detect_locks_restart_manager`. The size-query/fill pattern uses `ERROR_MORE_DATA` (234), which the documentation lists for a too-small buffer; `RmEndSession` executes in a `finally` block so a session is never leaked.

Three properties matter for interpretation. First, Restart Manager reports what it determines is using the registered resource; it is an installer-oriented interface, not documented as a universal enumeration of all handles, so an empty list means only that no holder was reported. Second, the query can fail. In the live evidence run, `RmGetList` returned Win32 error 5

(ERROR_ACCESS_DENIED) for the permission-restriction fixture, and the analyzer reported an inspection error. Third, the documentation does not explain that observation: Microsoft lists error 5 for RmGetList as "a path registered to the Restart Manager session is a directory" [11], which does not describe this file target, and the analyzer's own directory check runs before Restart Manager is called. The report therefore records the error code as observed and does not claim that the deny ACE is the documented cause (Section 6.8, Scenario 05).

3.4 Permission and ownership evidence

os.access() is the access-check source. As recorded in the project's engineering log, it tests basic user rights and the read-only flag without evaluating recursive NTFS DACL inheritance or directory-level deletion permissions, and the Python documentation cautions that real I/O may fail even when it reports access [15]. Its output is therefore preliminary access evidence, not a prediction of the real operation. Scenario 05 illustrates the boundary: for a file carrying an explicit deny ACE, the Permissions panel still lists READ as allowed, while the Restart Manager query failed with Win32 error 5. Windows evaluates ACEs against the caller's token and stops at the first explicit deny [26]; the Diagnoser does not. The GUI states "full NTFS DACL not evaluated" beside the detection method. The owner lookup identifies who owns the file but does not by itself explain permissions.

3.5 File attribute semantics

Table 3.3. Win32 attribute flags decoded by FileAttributeAnalyzer [27]

Flag	Value	Treatment in the Diagnoser
FILE_ATTRIBUTE_READONLY	0x0001	Microsoft: applications can read the file but cannot write to or delete it. Diagnosis: blocks WRITE and DELETE (Rule 3); does not block READ or EXECUTE
FILE_ATTRIBUTE_HIDDEN	0x0002	Informational only; never a blocker
FILE_ATTRIBUTE_SYSTEM	0x0004	Informational only; never a blocker
FILE_ATTRIBUTE_ARCHIVE	0x0020	Microsoft: typically used to mark files for backup or removal. Informational only
FILE_ATTRIBUTE_REPARSE_POINT	0x0400	Microsoft: a file or directory with an associated reparse point, or a symbolic link. Displayed; the Cloud-Sync Analyzer uses it as a placeholder heuristic (potential_contention)
FILE_ATTRIBUTE_OFFLINE	0x1000	Microsoft: data not immediately available, used by Remote Storage. Displayed; the Cloud-Sync Analyzer treats it (with the recall-on-data-access bit) as a placeholder heuristic

3.6 Cloud-sync discovery sources

Table 3.4. How supported provider roots and clients are located

Provider	Root discovery	Client process names
Microsoft OneDrive	Environment variables `OneDrive`, `OneDriveConsumer`, `OneDriveCommercial`; registry `HKCU\Software\Microsoft\OneDrive` (account `UserFolder` values)	`onedrive.exe`, `filecoauth.exe`
Google Drive	Registry `DriveFS` mount-point values; mounted drives found through `psutil.disk_partitions` and `GetVolumeInformationW` (label or `DriveFS` file-system name)	`googledrivefs.exe`

Discovery is bounded to the local environment and registry configuration and may be incomplete; the GUI says so in the sync panel. Provider documentation describes how OneDrive and Google Drive present files locally [4, 5]; it does not establish that a given file has a live lock. A client process found running through `psutil` is a hint of possible contention, not proof of a conflict.

3.7 Platform handling

Windows is the supported platform. On other platforms `ProcessLockAnalyzer` returns an `unsupported_platform` result, and the attribute and permission analyzers fall back to portable `os.stat` / `os.access` behaviour so that the rest of the package can be developed and tested anywhere. Windows-specific tests are marked `skipif(sys.platform != "win32")`; the one test that checks the unsupported-platform message is the mirror image and is skipped on Windows (Section 6.6).

3.8 Source-code organisation

The application is a Python package `diagnoser` under `src/`, with tests under `tests/` and evidence and demonstration scripts under `scripts/`. The line counts below include docstrings and comments.

Table 3.5. Application source files

File	Lines	Role
`models.py`	278	Enums and result dataclasses
`analyzers/base.py`	54	`BaseAnalyzer` abstract class; path normalisation helpers
`analyzers/process_lock.py`	352	`ProcessLockAnalyzer` — Restart Manager lock detection
`analyzers/permissions.py`	366	`PermissionAnalyzer` — access checks and owner resolution
`analyzers/file_attributes.py`	240	`FileAttributeAnalyzer` — attribute decoding
`analyzers/sync_conflict.py`	596	`SyncConflictAnalyzer`, `correlate_sync_lock`, `is_subpath`
`engine/diagnosis.py`	359	`DiagnosisEngine` — orchestration, classification, secondary notes
`recommendations/recommender.py`	252	`Recommender` — advisory guidance
`gui/main_window.py`	1,145	`MainWindow` — Tkinter presentation layer
`gui/app.py`	23	Entry point (`python -m diagnoser.gui.app`)

4. DESIGN

4.1 Development methodology

The project followed an incremental, evidence-driven method. Each milestone added one capability with its own structured result type and automated tests, and a living engineering log recorded design decisions, test totals and limitations. The Review 2 prototype contained the Process / Lock Analyzer, the Permission / Ownership Analyzer, a three-step Diagnosis Engine and the Tkinter GUI. Review 3 completed the system in five sprints followed by an evidence-completion phase.

Table 4.1. Review 3 development milestones (from the engineering log)

Milestone	Principal additions	Principal files
Sprint 1	Extended data models; File Attribute Analyzer; standalone Recommender	`models.py`, `file_attributes.py`, `recommender.py`
Sprint 2	Cloud-Sync Analyzer for OneDrive / Google Drive; conservative evidence tiers; `correlate_sync_lock`	`sync_conflict.py`
Sprint 3	Diagnosis Engine coordinating all four peers; deterministic priority classification; secondary-evidence notes; analyzer-exception isolation	`engine/diagnosis.py`
Sprint 4	GUI cards for attributes and cloud sync; `_reset_panels()` against stale data; DACL-limitation disclaimer	`gui/main_window.py`
Sprint 5 and evidence phase	Automated live GUI capture; reusable fixtures; lock-holding helper; coverage matrix and screenshot manifest	`scripts/`, evidence pack

4.2 System architecture

The final architecture has seven logical elements. The Tkinter GUI collects input and renders output. The Diagnosis Engine validates input, invokes the analyzers and coordinates the result. Four peer analyzers gather evidence independently. Their structured outputs are collected and correlated, a classification step selects the root cause, the Recommendation Engine produces advisory guidance, and the final result is displayed in the GUI.

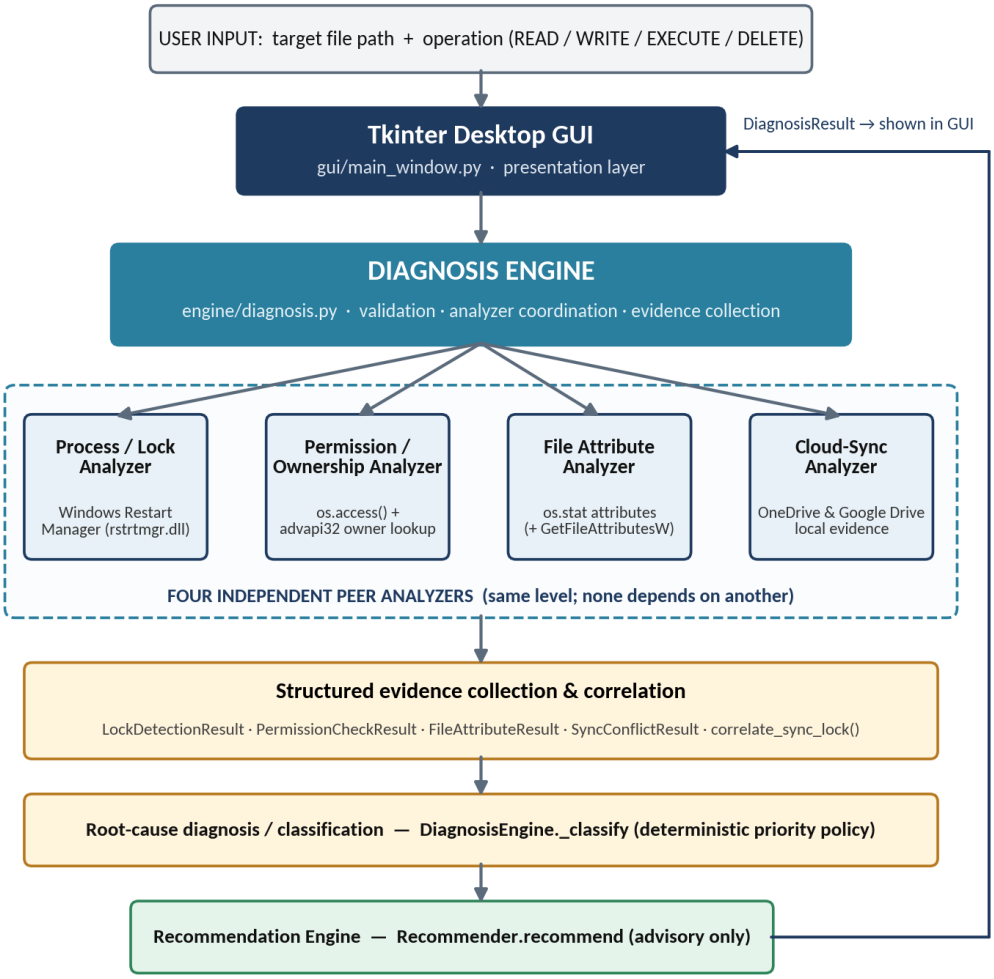


Figure 4.1. Final system architecture. The four analyzers are peers beneath the Diagnosis Engine: none is a child of, or depends on, another. Evidence flows upward as structured result objects; the classified and annotated `DiagnosisResult` returns to the GUI.

Table 4.2. Architectural elements and their implementing code

Element	Responsibility	Implementation
Tkinter desktop GUI	Collect path and operation; invoke the engine; render status, process list, permissions, attributes, sync and recommendation	`gui/main_window.py` (`MainWindow`), `gui/app.py`
Diagnosis Engine	Normalise input, handle missing and directory targets, call the four analyzers through guarded wrappers, correlate, classify, assemble the result	`engine/diagnosis.py` (`DiagnosisEngine`)
Process / Lock Analyzer	Report processes holding the file through Windows Restart Manager	`analyzers/process_lock.py` (`ProcessLockAnalyzer`)
Permission / Ownership Analyzer	Report access results per operation and the file owner	`analyzers/permissions.py` (`PermissionAnalyzer`)
File Attribute Analyzer	Read-only inspection of Win32 attributes	`analyzers/file_attributes.py` (`FileAttributeAnalyzer`)
Cloud-Sync Analyzer	Local OneDrive / Google Drive association and conservative evidence level; lock correlation helper	`analyzers/sync_conflict.py` (`SyncConflictAnalyzer`, `correlate_sync_lock`)
Evidence collection and correlation	Typed result objects attached to one `DiagnosisResult`; sync-lock correlation	`models.py`; `correlate_sync_lock()`

Element	Responsibility	Implementation
Root-cause classification	Deterministic priority policy	`DiagnosisEngine._classify()`
Recommendation Engine	Advisory text from the diagnosis and raw evidence	`recommendations/recommender.py` (`Recommender`)

4.3 Design principles

- **Peer decoupling.** Each analyzer extends `BaseAnalyzer`, returns its own dataclass and never calls another analyzer. The only cross-analyzer logic is the explicit `correlate_sync_lock()` contract, applied by the engine.
- **Structured evidence.** Analyzers return typed results (`LockDetectionResult`, `PermissionCheckResult`, `FileAttributeResult`, `SyncConflictResult`). The engine and GUI never call Windows APIs directly.
- **Isolation of failure.** The engine calls analyzers through `_safe_analyze_*` wrappers that convert unexpected exceptions into structured error results, so one failing analyzer cannot crash a diagnosis.
- **Conservative claims.** A diagnosis is asserted only when evidence supports it. Sync-folder membership or a running sync client is reported as association or potential contention, never as a confirmed conflict.
- **Read-only safety.** Analyzers inspect state and never modify files, attributes, permissions, processes or the registry.
- **Testability.** Analyzers and the recommender can be injected into `DiagnosisEngine`, and `SyncConflictAnalyzer` accepts override hooks for roots, processes and attributes.

4.4 Data model

All components communicate through dataclasses defined in `models.py`. `FileOperation` enumerates the four operations and `DiagnosisType` the seven diagnosis categories. `DiagnosisResult` carries the primary diagnosis, the reason, the suggested action and the four raw analyzer results, which is what allows the GUI to show primary and secondary evidence together.

src/diagnoser/models.py lines 17, 22–25, 205, 211–217

```
 17  class FileOperation(Enum):
     ⋮
 22      READ = "read"
 23      WRITE = "write"
 24      EXECUTE = "execute"
 25      DELETE = "delete"
     ⋮
205  class DiagnosisType(Enum):
     ⋮
211      FILE_NOT_FOUND = "file_not_found"
212      FILE_LOCKED = "file_locked"
213      PERMISSION_DENIED = "permission_denied"
214      READ_ONLY = "read_only"
215      OPERATION_ALLOWED = "operation_allowed"
216      UNKNOWN_ERROR = "unknown_error"
217      SYNC_CONFLICT = "sync_conflict"
```

Listing 1. The operation and diagnosis enumerations. Seven diagnosis categories exist, including `SYNC_CONFLICT`, which was added in Review 3.

src/diagnoser/models.py lines 220–221, 238–255

```
220  @dataclass
221  class DiagnosisResult:
     ⋮
238                              was performed.  None otherwise.
239          sync_result: The raw SyncConflictResult, if a sync check
240                       was performed.  None otherwise.
241      """
242      file_path: str
243      operation: str = ""
244      diagnosis: DiagnosisType = DiagnosisType.UNKNOWN_ERROR
245      reason: str = ""
246      suggested_action: str = ""
247      lock_result: Optional[LockDetectionResult] = None
248      permission_result: Optional[PermissionCheckResult] = None
249      attribute_result: Optional[FileAttributeResult] = None
250      sync_result: Optional[SyncConflictResult] = None
251
252
253  # ============================================================================
254  # Generic Analyzer Result (used by future modules)
255  # ============================================================================
```

Listing 2. Fields of `DiagnosisResult` (docstring lines omitted). The four `*_result` fields attach every peer analyzer's raw evidence to the primary diagnosis.

Table 4.3. Structured result types

Dataclass	Produced by	Key fields
`ProcessInfo`	`ProcessLockAnalyzer`	`process_name`, `pid`, `executable_path`
`LockDetectionResult`	`ProcessLockAnalyzer`	`is_locked`, `processes`, `error`, `detection_method`
`PermissionCheckResult`	`PermissionAnalyzer`	`operation`, `permitted`, `owner`, `current_user`, `permission_bits`, `detection_method`
`FileAttributeResult`	`FileAttributeAnalyzer`	`is_readonly`, `is_hidden`, `is_system`, `is_archive`, `is_reparse_point`, `is_offline`, `raw_attributes`
`SyncConflictResult`	`SyncConflictAnalyzer`	`provider`, `sync_root`, `client_running`, `evidence_level`, `is_conflict`, `locking_process`
`DiagnosisResult`	`DiagnosisEngine`	`diagnosis`, `reason`, `suggested_action`, plus the four results above

4.5 End-to-end workflow

A diagnosis begins when the user presses Diagnose File and ends when every GUI panel has been refreshed. Figure 4.2 summarises the pipeline implemented by `MainWindow._run_diagnosis()` and `DiagnosisEngine.diagnose()`.

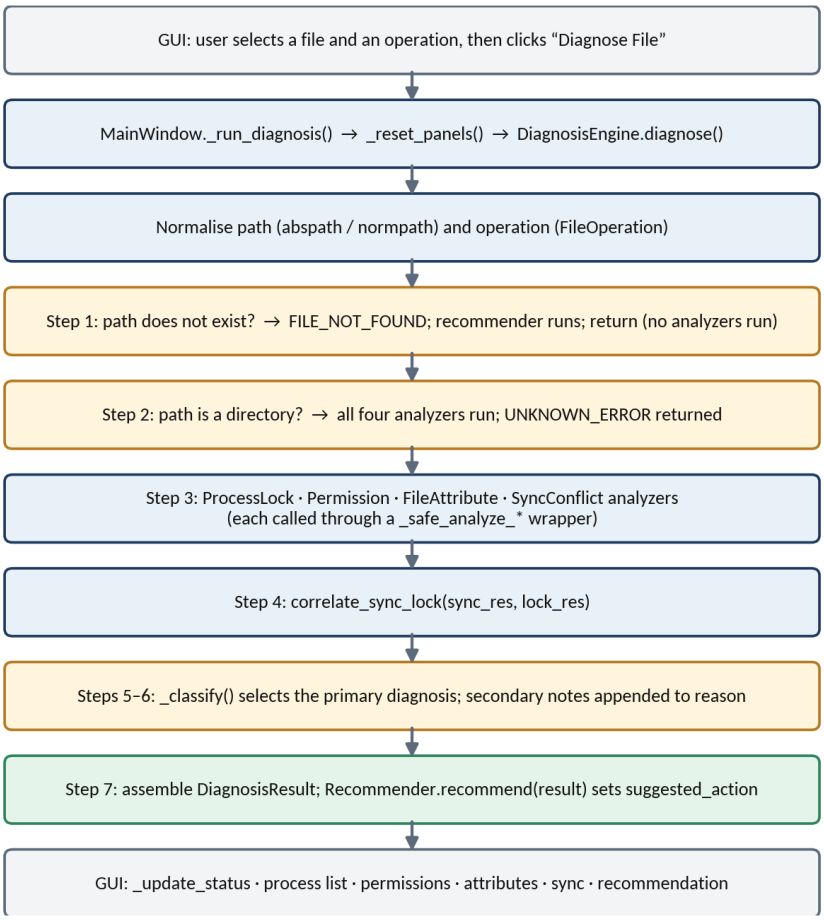


Figure 4.2. Diagnosis pipeline from user input to the displayed result. Steps 1 and 2 are early exits inside `diagnose()`; Steps 3–7 run for every existing regular file.

1. **Input and validation (GUI).** The user enters or browses to a path and selects READ, WRITE, EXECUTE or DELETE. An empty path produces a warning dialog; an unrecognised operation produces an error dialog. Neither starts a diagnosis.
2. **Reset.** `_reset_panels()` returns every panel to its neutral state and the status badge shows “ANALYZING…”, so no result from a previous run can remain on screen. Panels for checks that are skipped or fail explicitly report that state (for example, “lock analysis skipped” or “Unknown / Unverified”) rather than leaving an ambiguous blank that could look like success.
3. **Normalisation (engine).** The path is converted with `os.path.abspath(os.path.normpath(...))` and the operation string is mapped to a `FileOperation` value.
4. **Missing target.** If the path does not exist the engine returns `FILE_NOT_FOUND` immediately; no analyzer runs and the GUI marks each downstream panel as skipped.

5. **Directory target.** If the path is a directory all four analyzers still run, and the engine returns `UNKNOWN_ERROR` with the reason “The specified path is a directory, not a regular file.”
6. **Peer analysis.** For an existing regular file the four analyzers run through guarded wrappers and each returns a structured result.
7. **Correlation.** `correlate_sync_lock()` checks whether any holding process matches a supported sync client and, if so, upgrades the sync evidence to `confirmed_lock`.
8. **Classification and secondary findings.** `_classify()` selects the primary diagnosis; secondary notes are appended to the reason; the full evidence stays attached to the result.
9. **Recommendation.** `Recommender.recommend()` produces the advisory text from the classified result.
10. **Display.** The GUI populates the status, process-list, permission, attribute, sync and recommendation panels from the `DiagnosisResult`.

4.6 Diagnosis-priority rules

Several conditions can hold at once — for example a read-only file that is also held open by another program. The engine therefore applies an ordered rule list in which the first matching rule decides the primary diagnosis. Figure 4.3 shows the order implemented in `DiagnosisEngine._classify()`, and Listing 3 is the corresponding source.

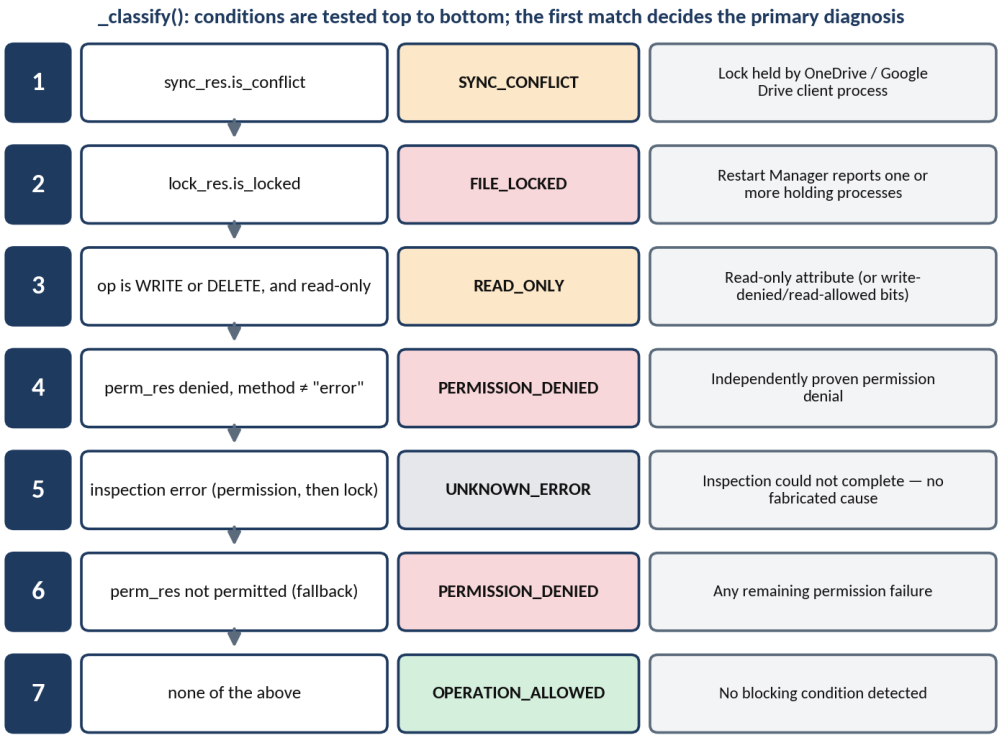


Figure 4.3. Order of evaluation in `_classify()`. The first matching condition decides the primary diagnosis. FILE_NOT_FOUND and the directory case are handled earlier in `diagnose()`.

src/diagnoser/engine/diagnosis.py lines 267–311

```
        """Classify primary failure root cause according to deterministic priority."""
        # 1. Confirmed sync conflict lock (holding process matches OneDrive or Google Drive)
        if sync_res and sync_res.is_conflict:
            return DiagnosisType.SYNC_CONFLICT, self._build_sync_conflict_reason(sync_res)

        # 2. Confirmed application process lock (Restart Manager detected active handles)
        if lock_res and lock_res.is_locked:
            return DiagnosisType.FILE_LOCKED, self._build_lock_reason(lock_res)

        # 3. Read-only restriction (relevant for WRITE and DELETE)
        if operation in (FileOperation.WRITE, FileOperation.DELETE):
            is_ro = False
            if attr_res and attr_res.is_readonly is True:
                is_ro = True
            elif perm_res and not perm_res.permitted and perm_res.detection_method != "error":
                bits = perm_res.permission_bits or {}
                if bits.get("read", False) and not bits.get("write", False):
                    is_ro = True

            if is_ro:
                reason = (
                    f"The file is read-only. The current user cannot "
                    f"{operation.value} this file."
                )
                return DiagnosisType.READ_ONLY, reason

        # 4. Proven permission denial (independent verified evidence from Permission Analyzer)
        if perm_res and not perm_res.permitted and perm_res.detection_method != "error":

return DiagnosisType.PERMISSION_DENIED, self._build_permission_reason(operation, perm_res)

        # 5. Critical inspection failures (precede success when inspection was blocked/
impossible)
        if perm_res and perm_res.error and perm_res.detection_method == "error":
            return DiagnosisType.UNKNOWN_ERROR, f"Permission inspection error: {perm_res.error}"
        if lock_res and lock_res.error and lock_res.detection_method == "error":
            return DiagnosisType.UNKNOWN_ERROR, f"Lock inspection error: {lock_res.error}"

        # 6. Fallback permission check (if any other permission failure was reported)
        if perm_res and not perm_res.permitted:

return DiagnosisType.PERMISSION_DENIED, self._build_permission_reason(operation, perm_res)

        # 7. No blocking condition detected
        return (
            DiagnosisType.OPERATION_ALLOWED,
            "The requested operation is permitted and no lock was detected.",
        )
```

Listing 3. The complete deterministic classification policy (`DiagnosisEngine._classify`). Rules are numbered as in the source comments.

Table 4.4. Rule rationale

Rule	Condition (source)	Result	Why it sits here
1	`sync_res.is_conflict` (set only by `correlate_sync_lock` when a holding process is a supported sync client)	`SYNC_CONFLICT`	More specific than a generic lock: the holder is a known provider client, so provider-specific advice applies.
2	`lock_res.is_locked`	`FILE_LOCKED`	A live handle is direct runtime evidence and can block an otherwise permitted, writable file, so it precedes static properties.
3	WRITE or DELETE and the read-only attribute (or write denied while read is allowed)	`READ_ONLY`	A specific, user-fixable cause. It is evaluated only for operations it actually blocks, so READ and EXECUTE are unaffected.
4	Permission result not permitted and `detection_method != "error"`	`PERMISSION_DENIED`	Independently proven denial from the Permission Analyzer.
5	Permission inspection error, then lock inspection error	`UNKNOWN_ERROR`	If a check could not complete, the engine must not report success or invent a cause.
6	Permission result not permitted (any other case)	`PERMISSION_DENIED`	Fallback so no denial is silently dropped.
7	No condition matched	`OPERATION_ALLOWED`	No blocking condition was detected by the checks that ran.

Rule numbers in this report follow the order of conditions inside `_classify()`. The evidence package refers to the branch that produced the blocked permission scenario as “priority rule 4”; in the code it is the lock-inspection-error branch of Rule 5 above.

4.7 Why a deterministic policy, and why secondary evidence is preserved

A deterministic rule list was chosen over weighted scoring for four reasons. It is reproducible: the same evidence always yields the same diagnosis, which makes the behaviour testable with injected analyzer results. It is explainable: each outcome can be traced to one named condition. It is honest: no numeric confidence is invented where the operating system provides only presence or absence of evidence. And it is specificity-first: the more specific and more actionable cause is reported ahead of a generic one.

A single primary label cannot describe every coexisting finding, so the engine never discards the rest. All four raw results stay attached to `DiagnosisResult`, the GUI renders every panel regardless of the headline, and `diagnose()` appends short secondary notes to the reason. This avoids overstating the root cause: a locked file that is also read-only is reported as locked, with the read-only attribute still visible; a file inside a OneDrive folder is reported as “located in OneDrive folder”, not as a sync conflict.

src/diagnoser/engine/diagnosis.py lines 167–190

```
167          secondary_notes = []
168          if diagnosis_type != DiagnosisType.SYNC_CONFLICT and sync_res:
169              prov = PROVIDER_DISPLAY_NAMES.get(
170 (sync_res.provider or "").lower(), (sync_res.provider or "cloud sync").capitalize()
171              )
172              if sync_res.evidence_level == "potential_contention":
173                  secondary_notes.append(f"Located in {prov} folder with client running")
174              elif sync_res.is_synced:
175                  secondary_notes.append(f"Located in {prov} folder")
176
177          if diagnosis_type != DiagnosisType.READ_ONLY and attr_res and attr_res.is_readonly:
178              if operation in (FileOperation.WRITE, FileOperation.DELETE):
179                  secondary_notes.append("File also has the read-only attribute set")
180
181 if diagnosis_type not in (DiagnosisType.FILE_LOCKED, DiagnosisType.SYNC_CONFLICT) and lock_res and lock_
 res.is_locked:
182              secondary_notes.append("File is also locked by an active process")
183
184          if diagnosis_type == DiagnosisType.PERMISSION_DENIED and lock_res and lock_res.error:
185              secondary_notes.append(f"Lock inspection could not complete: {lock_res.error}")
186
187          if secondary_notes:
188              full_reason = f"{primary_reason} ({'; '.join(secondary_notes)}.)"
189          else:
190              full_reason = primary_reason
```

Listing 4. Secondary-context construction in `DiagnosisEngine.diagnose()`: notes are appended to the primary reason without changing the primary diagnosis.

4.8 Worked traces

Table 4.5. How selected scenarios resolve through the pipeline

Scenario	Evidence collected	Deciding step	Primary result and secondary evidence
Missing path (Scenario 02)	None — analyzers not run	Step 1 of `diagnose()`	`FILE_NOT_FOUND`; GUI panels report checks skipped
Normal file, WRITE (01b)	No lock; permission bits all allowed; attributes 0x0020 (archive); no sync association	Rule 7	`OPERATION_ALLOWED`
Read-only file, WRITE (04a)	No lock; attributes 0x0021; WRITE denied	Rule 3	`READ_ONLY`

Scenario	Evidence collected	Deciding step	Primary result and secondary evidence
Read-only file, READ (04b)	Same evidence as 04a	Rule 3 not entered (operation is READ); Rule 7	`OPERATION_ALLOWED`; read-only flag still displayed
Held and read-only, WRITE (08)	Restart Manager returns a holder; attributes 0x0021	Rule 2	`FILE_LOCKED`; reason gains "File also has the read-only attribute set"
Deny-read ACE (05)	Permission analyzer reports allowed (the DACL is not evaluated); `RmGetList` fails with Win32 error 5	Rule 5 (lock inspection error)	`UNKNOWN_ERROR` — expected `PERMISSION_DENIED` was not reproduced (BLOCKED)
Directory (09)	All analyzers return "not a file" errors	Step 2 of `diagnose()`	`UNKNOWN_ERROR`
OneDrive client holds the file (automated test)	Injected lock: `OneDrive.exe`; sync root matched	Rule 1	`SYNC_CONFLICT` — verified by `test_confirmed_onedrive_lock_returns_sync_conflict`, not by a live demonstration
File in sync folder, no lock (automated test)	Sync root matched; no confirmed lock	Rule 7	`OPERATION_ALLOWED`, secondary note "Located in … folder" — `test_sync_folder_path_only_does_not_falsely_return_sync_conflict`

4.9 GUI design

The GUI is a single dark-themed window built with Tkinter and `ttk`. The content sits in a scrollable canvas so that all six result panels remain usable on smaller screens. The user flow follows Review 1: select a file, choose an operation, press Diagnose File, and read the diagnosis with its supporting evidence in one place.

Table 4.6. GUI panels and the result data they display

Panel	Content	Updated by
Input area	File-path entry, Browse button, operation selector (READ, WRITE, EXECUTE, DELETE), Diagnose File button	`_build_ui`, `_browse_file`
Status	Colour-coded diagnosis badge, file path, operation and the reason (including secondary notes)	`_update_status`
Process List	Process name, PID and executable path of each holder; messages for "no locks", skipped or failed inspection	`_update_process_list`
Permissions	Owner, current user, detection method with the DACL-not-evaluated note, and Allowed / Denied / Unknown for READ, WRITE, EXECUTE, DELETE	`_update_permissions`
File Attributes	Raw bitmask, detection method, operation-aware effect text, and six flags	`_update_attributes`
Cloud Synchronization	Provider, sync root, client daemon, evidence level and detail text	`_update_sync`
Recommendation	Advisory text from the Recommendation Engine	`_update_recommendation`

5. PROPOSED SYSTEM

5.1 Process / Lock Analyzer

Aspect	Description
Purpose	Identify which processes currently hold a file open.
Input	Target file path.
Processing	Validates the path (missing, directory, no read access → structured error). On Windows it calls the Restart Manager: start a session, register the file, call `RmGetList` once to learn the required buffer size and again to fill an `RM_PROCESS_INFO` array, then always end the session. Each entry is converted to `ProcessInfo`; `psutil` is used only to look up the executable path from the PID.
Output	`LockDetectionResult` with `is_locked`, a list of `ProcessInfo`, an `error` string if inspection failed, and `detection_method` (`restart_manager`, `unsupported_platform` or `error`).
Contribution	Highest-priority runtime evidence. A holder leads to `FILE_LOCKED` (or `SYNC_CONFLICT` after correlation); an inspection error can lead to `UNKNOWN_ERROR` when nothing else explains the state.

Windows Restart Manager is the primary and only lock-detection mechanism. It was designed for installers: files in use by running applications or services are the main reason an update needs a restart [24]. A session registers resources such as file names [10], and `RmGetList` returns the applications and services currently using the registered resources [11]; Microsoft's function overview describes it as used by installers to list the applications affected by those resources [25]. The result is therefore evidence of what Restart Manager reports for the one registered file. It is not documented as an enumeration of every open handle on the system (compare Sysinternals Handle [6]), so the analyzer does not treat an empty list as proof that nothing holds the file. Listing 5 shows the session start [9] and resource registration; Listing 6 shows the two-call buffer pattern and the guaranteed cleanup with `RmEndSession` [12].

src/diagnoser/analyzers/process_lock.py lines 209–236

```
209          session_handle = wintypes.DWORD()
210          session_key = ctypes.create_unicode_buffer(CCH_RM_SESSION_KEY + 1)
211
212          # --- Step 1: Start a Restart Manager session ---
213          ret = _rstrtmgr.RmStartSession(
214              ctypes.byref(session_handle), 0, session_key
215          )
216          if ret != ERROR_SUCCESS:
217              return LockDetectionResult(
218                  file_path=file_path,
219                  is_locked=False,
220                  error=f"RmStartSession failed with error code {ret}.",
221                  detection_method="error",
222              )
223
224          try:
225              # --- Step 2: Register the file as a resource ---
226              file_path_ptr = ctypes.c_wchar_p(file_path)
227              files_array = (ctypes.c_wchar_p * 1)(file_path_ptr)
228              ret = _rstrtmgr.RmRegisterResources(
229                  session_handle,
230                  1,                  # nFiles
231                  files_array,
232                  0,                  # nApplications
233                  None,               # rgApplications
234                  0,                  # nServices
235                  None,               # rgsServiceNames
236              )
```

Listing 5. Starting a Restart Manager session and registering the target file (`ProcessLockAnalyzer._detect_locks_restart_manager`).

src/diagnoser/analyzers/process_lock.py lines 245–274, 307–309

```
            # --- Step 3: Get the list of processes using this file ---
            pn_proc_info_needed = wintypes.UINT(0)
            pn_proc_info = wintypes.UINT(0)
            reboot_reasons = wintypes.DWORD(0)

            # First call: ask how large the buffer needs to be
            ret = _rstrtmgr.RmGetList(
                session_handle,
                ctypes.byref(pn_proc_info_needed),
                ctypes.byref(pn_proc_info),
                None,
                ctypes.byref(reboot_reasons),
            )

            if ret == ERROR_SUCCESS and pn_proc_info_needed.value == 0:
                # Case 2: no processes are using this file
                return LockDetectionResult(
                    file_path=file_path,
                    is_locked=False,
                    detection_method="restart_manager",
                )

            if ret != ERROR_MORE_DATA:
                # Unexpected error (e.g. access denied)
                return LockDetectionResult(
                    file_path=file_path,
                    is_locked=False,
                    error=f"RmGetList failed with error code {ret}.",
                    detection_method="error",
                )
    ⋮
        finally:
            # --- Step 4: Always clean up the session ---
            _rstrtmgr.RmEndSession(session_handle)
```

Listing 6. Querying the holder list. The first `RmGetList` call returns `ERROR_MORE_DATA` (234) with the needed size; any other failure, such as access denied, is returned as a structured error. `RmEndSession` runs in `finally`.

`RmGetList` fills one `RM_PROCESS_INFO` record per application or service, containing the process ID and start time and an application name. The analyzer stores the application name as `process_name`, which is why the live capture shows “Python” rather than a full image name. The executable path is added afterwards.

src/diagnoser/analyzers/process_lock.py lines 332–350

```
        for i in range(count):
            pid = rm_infos[i].Process.dwProcessId
            app_name = rm_infos[i].strAppName or ""

            # Enrich with the full executable path via psutil
            exe_path = ""
            try:
                import psutil
                proc = psutil.Process(pid)
                exe_path = proc.exe() or ""
            except Exception:
                # Process may have exited, or access denied — leave blank
                pass

            results.append(ProcessInfo(
                process_name=app_name,
                pid=pid,
                executable_path=exe_path,
            ))
```

Listing 7. `psutil` enrichment in `ProcessLockAnalyzer._extract_process_info`. A failure to resolve the path (process exited, access denied) leaves the path empty; it never changes the lock result.

5.2 Permission / Ownership Analyzer

Aspect	Description
Purpose	Report whether the current user has access for the selected operation, show all four access results, and identify the file owner.
Input	Target file path and `FileOperation`.

Aspect	Description
Processing	Validates the path; determines the current user with `getpass.getuser()`; maps the operation to an `os.access()` flag (READ → `R_OK`, WRITE → `W_OK`, EXECUTE → `X_OK`, DELETE → `W_OK`); computes read/write/execute/delete bits; resolves the owner with `GetNamedSecurityInfoW` and `LookupAccountSidW`.
Output	`PermissionCheckResult` with `permitted`, `permission_bits`, `owner`, `current_user`, `details` and `detection_method` (`windows_acl` when an owner was resolved, else `os_access`).
Contribution	Feeds the permission-denial rules and the read-only inference in `_classify()`, supplies the owner name used in advice, and provides the four rows of the Permissions panel.

src/diagnoser/analyzers/permissions.py lines 81–89, 156–159, 170–174, 182–186

```
 81 _OPERATION_TO_ACCESS_FLAG = {
 82     FileOperation.READ: os.R_OK,
 83     FileOperation.WRITE: os.W_OK,
 84     FileOperation.EXECUTE: os.X_OK,
 85     # DELETE on POSIX requires write on the parent directory;
 86     # on Windows os.access doesn't directly test delete, so we
 87     # approximate with write-check and note the limitation.
 88     FileOperation.DELETE: os.W_OK,
 89 }
156         # ----- Cross-platform permission check via os.access() -----
157         try:
158             access_flag = _OPERATION_TO_ACCESS_FLAG[operation]
159             permitted = os.access(file_path, access_flag)
170         # ----- Gather all permission bits -----
171         permission_bits = self._get_permission_bits(file_path)
172
173         # ----- Get file owner -----
174         owner = self._get_file_owner(file_path)
182         # ----- Choose detection method label -----
183         if _IS_WINDOWS:
184             detection_method = "windows_acl" if owner else "os_access"
185         else:
186             detection_method = "os_access"
```

Listing 8. Operation-to-flag mapping and the `os.access()` preliminary access check in `permissions.py`. DELETE is approximated by write access, as the source comment notes.

src/diagnoser/analyzers/permissions.py lines 270–285, 294–311

```
270         p_sid_owner = ctypes.c_void_p()
271         p_sd = ctypes.c_void_p()  # security descriptor (must be freed)
272
273         try:
274             ret = _advapi32.GetNamedSecurityInfoW(
275                 file_path,
276                 SE_FILE_OBJECT,
277                 OWNER_SECURITY_INFORMATION,
278                 ctypes.byref(p_sid_owner),  # ppsidOwner
279                 None,                        # ppsidGroup
280                 None,                        # ppDacl
281                 None,                        # ppSacl
282                 ctypes.byref(p_sd),          # ppSecurityDescriptor
283             )
284             if ret != ERROR_SUCCESS:
285                 return ""
294             success = _advapi32.LookupAccountSidW(
295                 None,                       # lpSystemName (local)
296                 p_sid_owner,                # Sid
297                 name_buf,                   # Name
298                 ctypes.byref(name_len),     # cchName
299                 domain_buf,                 # ReferencedDomainName
300                 ctypes.byref(domain_len),   # cchReferencedDomainName
301                 ctypes.byref(sid_name_use), # peUse
302             )
303
304             if not success:
305                 return ""
306
307             domain = domain_buf.value
308             name = name_buf.value
309             if domain:
310                 return f"{domain}\\{name}"
311             return name
```

Listing 9. Owner resolution on Windows. Only `OWNER_SECURITY_INFORMATION` is requested and the DACL pointer is `None`: the security descriptor's DACL is not retrieved or evaluated.

`GetNamedSecurityInfoW` retrieves a copy of an object's security descriptor; the owner SID it returns is valid only when `OWNER_SECURITY_INFORMATION` is requested, the DACL pointer only when `DACL_SECURITY_INFORMATION` is requested, and the returned buffer must be released with `LocalFree` [13]. `LookupAccountSidW` converts the SID into an account and domain name [14]. The analyzer requests the owner only, so no DACL is retrieved or evaluated, regardless of the unused DACL constant in the file.

The distinction matters because Windows decides access by comparing the ACEs in an object's DACL with the caller's access token and stopping at the first explicit deny or sufficient allow [26]. The Diagnoser does not perform that evaluation. Its `os.access()` result is preliminary access evidence; the Python documentation itself cautions that I/O operations may fail even when `access()` indicates they would succeed [15]. It is not a prediction that the requested operation will succeed or fail, and it is never a substitute for attempting the operation. The GUI repeats this boundary beside every detection-method label.

5.3 File Attribute Analyzer

Aspect	Description
Purpose	Expose file-system attributes that can affect an operation or explain a file's behaviour.
Input	Target file path.
Processing	Reads `os.stat(path).st_file_attributes` as the primary source; if that value is unavailable it falls back to `GetFileAttributesW` through `ctypes`. The bitmask is decoded into six flags and a list of attribute names. A portable POSIX fallback exists for development on other platforms.
Output	`FileAttributeResult`: `is_readonly`, `is_hidden`, `is_system`, `is_archive`, `is_reparse_point`, `is_offline`, `raw_attributes`, `attributes_list`, `detection_method` (`windows_stat`, `posix_stat`, `error`).
Contribution	Read-only evidence for Rule 3; raw bitmask and per-flag display in the Attributes panel; hidden, system and archive flags are shown as information and never treated as blockers.

src/diagnoser/analyzers/file_attributes.py lines 153–169, 178–191

```
        # 1. Primary inspection: os.stat().st_file_attributes
        try:
            st = os.stat(file_path)
            raw_attrs = getattr(st, "st_file_attributes", None)
        except PermissionError:
            raise
        except OSError:
            pass

        # 2. Secondary fallback: kernel32.GetFileAttributesW via ctypes
        if raw_attrs is None and _kernel32 is not None:
            try:
                ret = _kernel32.GetFileAttributesW(ctypes.c_wchar_p(file_path))
                if ret != INVALID_FILE_ATTRIBUTES:
                    raw_attrs = int(ret)
            except Exception:
                pass
⋮
        # Decode individual standard bit flags
        is_readonly = bool(raw_attrs & FILE_ATTRIBUTE_READONLY)
        is_hidden = bool(raw_attrs & FILE_ATTRIBUTE_HIDDEN)
        is_system = bool(raw_attrs & FILE_ATTRIBUTE_SYSTEM)
        is_archive = bool(raw_attrs & FILE_ATTRIBUTE_ARCHIVE)
        is_reparse_point = bool(raw_attrs & FILE_ATTRIBUTE_REPARSE_POINT)
        is_offline = bool(raw_attrs & FILE_ATTRIBUTE_OFFLINE)

        # Build list of active attribute names
        attributes_list: List[str] = [
            name
            for mask, name in _ATTRIBUTE_FLAG_NAMES
            if (raw_attrs & mask)
        ]
```

Listing 10. Attribute extraction and decoding in `FileAttributeAnalyzer._analyze_windows`. The analysis is strictly read-only.

On Windows, `os.stat().st_file_attributes` is attempted first [16]; if it is unavailable the analyzer falls back to `GetFileAttributesW`, which returns the attribute DWORD or `INVALID_FILE_ATTRIBUTES` on failure [17]. The bit definitions follow Microsoft's documentation, which describes a read-only file as one that applications can read but cannot write to or delete [27]. The decoded flags drive operation-aware text in the GUI: a read-only file is described as blocking WRITE and DELETE but not READ.

5.4 Cloud-Sync Analyzer

Aspect	Description
Purpose	Determine whether the file is associated with a supported sync provider and how strong the evidence of synchronization contention is.
Input	Target file path (analysis), and the lock result (correlation step in the engine).
Processing	Discovers candidate OneDrive and Google Drive roots from environment variables, the registry, `psutil.disk_partitions` and `GetVolumeInformationW`; matches the file to the deepest root with a directory-boundary-safe, case-insensitive comparison (`is_subpath`); checks whether the provider's client process is running; checks reparse-point and offline placeholder attributes; assigns an evidence level.
Output	`SyncConflictResult`: `provider`, `sync_root`, `client_running`, `evidence_level`, `is_conflict`, `locking_process`, `details`.
Contribution	A confirmed conflict triggers Rule 1. Weaker evidence is shown in the Cloud Synchronization panel and as a secondary note, never as the primary diagnosis.

Table 5.1. Cloud-sync evidence tiers

Level	Meaning (as implemented)	Primary diagnosis?
`none`	File is not inside any detected OneDrive or Google Drive root. GUI text: “No provider association detected”.	No
`path_only`	File is inside a sync root, the client is not running, and no placeholder attributes are present.	No
`potential_contention`	File is inside a sync root and the client is running, or the file has reparse-point / offline placeholder attributes. “Active lock not confirmed”.	No
`confirmed_lock`	Restart Manager reported a holder whose process name matches the provider (`onedrive.exe`, `filecoauth.exe`, `googledrivefs.exe`).	Yes — `SYNC_CONFLICT`

src/diagnoser/analyzers/sync_conflict.py lines 341–349, 361–367

```
341          if not client_running and not (is_reparse or is_offline):
342              evidence_level = "path_only"
343              details = (
344                  f"File is located within a {provider_title} folder ({matched_root}), "
345                  f"but the {provider_title} client is not currently running. "
346                  "No active sync contention detected."
347              )
348          elif not client_running and (is_reparse or is_offline):
349              evidence_level = "potential_contention"
    ⋮
361          elif client_running and not (is_reparse or is_offline):
362              evidence_level = "potential_contention"
363              details = (
364                  f"File is located within an active {provider_title} sync folder ({matched_root}) "
365                  f"and the {provider_title} client process is running. Background synchronization "
366                  "may cause transient access contention, but no file lock is confirmed."
367              )
```

Listing 11. Conservative evidence-level assignment in `SyncConflictAnalyzer.analyze`. A running client alone yields `potential_contention`, with the explicit note that no file lock is confirmed.

src/diagnoser/analyzers/sync_conflict.py lines 142–157, 170–180

```
142      if not lock_result or not lock_result.is_locked or not lock_result.processes:
143          return sync_result
144
145      # Determine candidate sync process names
146      provider = sync_result.provider.lower()
147      candidate_procs = PROVIDER_PROCESS_NAMES.get(provider, set())
148
149      matching_proc: Optional[ProcessInfo] = None
150      inferred_provider = sync_result.provider
151
152      if candidate_procs:
153          for proc in lock_result.processes:
154              p_name = (proc.process_name or "").lower()
155              if p_name in candidate_procs:
156                  matching_proc = proc
157                  break
    ⋮
170      if matching_proc:
171          prov_title = PROVIDER_DISPLAY_NAMES.get(inferred_provider, "Cloud Sync")
172          return SyncConflictResult(
173              file_path=sync_result.file_path,
174              is_synced=sync_result.is_synced if sync_result.is_synced is not None else True,
175              provider=inferred_provider,
176              sync_root=sync_result.sync_root,
177              client_running=True,
178              evidence_level="confirmed_lock",
179              is_conflict=True,
180              locking_process=matching_proc,
```

Listing 12. Lock correlation in `correlate_sync_lock`. A lock is attributed to a provider only if a holder’s process name is in the provider’s known set; an unrelated holder (for example a text editor) is recorded as a non-sync lock.

Provider documentation explains how synchronised files can exist locally: OneDrive keeps a synced folder and can mark files as online-only or locally available [4], and Google Drive for desktop either streams files from a virtual drive or mirrors them into a local folder [5]. This is why the analyzer looks for sync roots, DriveFS volumes and placeholder attributes. It does not show that any particular file is locked: only a Restart Manager holder whose process name

matches a provider client produces a confirmed conflict. Likewise, the reparse-point and offline attributes are used as a project heuristic; Microsoft defines them generally and does not state that they identify OneDrive or Google Drive placeholders [27].

The analyzer does not call the Process / Lock Analyzer. Correlation is a separate, testable function that the engine applies to the two independent results, which preserves the peer relationship shown in Figure 4.1. Only Microsoft OneDrive and Google Drive are supported; other providers are out of scope.

5.5 Diagnosis Engine and evidence correlation

Aspect	Description
Purpose	Coordinate the four analyzers and produce one classified, annotated result.
Input	File path and operation (string or `FileOperation`); optional injected analyzers and recommender.
Processing	Normalise; handle missing and directory targets; run the analyzers through `_safe_analyze_*`; correlate sync and lock evidence; classify; build secondary notes; call the recommender.
Output	`DiagnosisResult` containing the primary `DiagnosisType`, reason, suggested action and all four raw results.
Contribution	The single point where evidence is interpreted. Analyzers report facts; only the engine decides what they mean together.

src/diagnoser/engine/diagnosis.py lines 151–164

```
151         # ── Step 3: Run All Four Peer Analyzers ──────────────────────
152         lock_res = self._safe_analyze_lock(file_path)
153         perm_res = self._safe_analyze_perm(file_path, operation)
154         attr_res = self._safe_analyze_attr(file_path)
155         sync_res = self._safe_analyze_sync(file_path)
156
157         # ── Step 4: Lock Correlation ─────────────────────────────────
158         # Correlate lock findings with sync provider findings
159         sync_res = correlate_sync_lock(sync_res, lock_res)
160
161         # ── Step 5: Deterministic Priority Classification ────────────
162         diagnosis_type, primary_reason = self._classify(
163             operation, lock_res, perm_res, attr_res, sync_res
164         )
```

Listing 13. Analyzer coordination, correlation and classification in `DiagnosisEngine.diagnose()`.

Each `_safe_analyze_*` wrapper catches any exception and returns the analyzer's result type with an error message and `detection_method="error"`. Automated tests confirm that an analyzer exception is isolated and represented honestly rather than crashing the diagnosis (Appendix A, Listing 18).

5.6 Recommendation Engine

Aspect	Description
Purpose	Convert the diagnosis and raw evidence into safe, human-readable advice.
Input	A `DiagnosisResult` (a `DiagnosisType`, dictionary or string is also accepted).

Aspect	Description
Processing	Dispatches on the diagnosis type and fills in details from the evidence, for example process names and PIDs for a lock, or the owner name for a permission denial.
Output	A recommendation string stored in `DiagnosisResult.suggested_action`.
Contribution	Turns evidence into a next step. The recommender is advisory: it never executes or triggers an action.

`src/diagnoser/recommendations/recommender.py` lines 57–76

```
57          if diag_type == DiagnosisType.FILE_NOT_FOUND:
58              return self._recommend_file_not_found()
59
60          elif diag_type == DiagnosisType.FILE_LOCKED:
61              return self._recommend_file_locked(lock_res)
62
63          elif diag_type == DiagnosisType.READ_ONLY:
64              return self._recommend_read_only(attr_res)
65
66          elif diag_type == DiagnosisType.PERMISSION_DENIED:
67              return self._recommend_permission_denied(perm_res)
68
69          elif diag_type == DiagnosisType.SYNC_CONFLICT:
70              return self._recommend_sync_conflict(sync_res, lock_res)
71
72          elif diag_type == DiagnosisType.OPERATION_ALLOWED:
73              return self._recommend_operation_allowed()
74
75          else:
76              return self._recommend_unknown()
```

Listing 14. Recommendation dispatch in `Recommender.recommend`.

`src/diagnoser/recommendations/recommender.py` lines 104–116

```
104     @staticmethod
105     def _recommend_file_locked(
106         lock_res: Optional[LockDetectionResult],
107     ) -> str:
108         if lock_res and lock_res.processes:
109             procs = lock_res.processes
110             if len(procs) == 1:
111                 p = procs[0]
112                 name = p.process_name or "(unknown process)"
113                 return (
114                     f"Save your work in {name} (PID {p.pid}) and close "
115                     "the application before retrying the operation."
116                 )
```

Listing 15. Evidence-grounded advice for a lock: the text names the holding process and PID returned by Restart Manager.

Table 5.2. Advisory guidance by diagnosis (summarised from `recommender.py`)

Diagnosis	Advice given
`FILE_NOT_FOUND`	Verify the path and that the file was not moved, renamed or deleted.
`FILE_LOCKED`	Save work in the named process (PID) and close it before retrying; several holders are listed together.
`READ_ONLY`	Review whether to clear the Read-only attribute in File Explorer › Properties before retrying.
`PERMISSION_DENIED`	Check NTFS permissions (Properties › Security) or contact the named owner or an administrator.
`SYNC_CONFLICT`	Wait for the provider to finish synchronizing, check its status, and optionally pause syncing; a path-only association is explicitly described as unconfirmed.
`OPERATION_ALLOWED`	No blocking condition detected.
`UNKNOWN_ERROR`	The cause cannot be determined from available evidence; ensure the path is accessible and consider appropriate privileges.

5.7 Tkinter GUI

Aspect	Description
Purpose	Collect input and present the structured result: primary diagnosis first, supporting evidence beside it.
Input	User-entered or browsed path; operation selected from READ, WRITE, EXECUTE, DELETE.
Processing	`_run_diagnosis()` validates input, resets all panels, calls the engine, and then calls `_update_status`, `_update_process_list`, `_update_permissions`, `_update_attributes`, `_update_sync` and `_update_recommendation`. Unexpected engine exceptions produce a “DIAGNOSIS ERROR” badge and a dialog.
Output	Status badge and reason; process table; owner, current user and four permission rows; raw bitmask, operational effect and six flags; provider, sync root, client state, evidence level and details; recommendation text.
Contribution	Makes all evidence visible, including secondary findings, and states analyzer boundaries (skipped checks, unevaluated DACL, unconfirmed sync).

src/diagnoser/gui/main_window.py lines 748–756, 762–764, 784–790

```
748          # Reliable reset before starting analysis
749          self._reset_panels()
750          self._status_badge.configure(
751              text="●  ANALYZING...",
752              background=_COLOURS["accent"],
753          )
754          self._status_file_lbl.configure(text=file_path)
755          self._status_op_lbl.configure(text=operation.value.upper())
756          self._status_reason_lbl.configure(text="Running diagnostic analyzers...")
⋮
762          # Execute diagnosis engine
763          try:
764              result = self._engine.diagnose(file_path, operation)
⋮
784          # Populate all panels from the DiagnosisResult
785          self._update_status(result)
786          self._update_process_list(result)
787          self._update_permissions(result)
788          self._update_attributes(result)
789          self._update_sync(result)
790          self._update_recommendation(result)
```

Listing 16. Core of `MainWindow._run_diagnosis`: validation, reset, engine call and panel population (selected lines).

src/diagnoser/gui/main_window.py lines 983–1004

```
983          op_str = (result.operation or "").lower()
984          if attr.error and attr.detection_method == "error":
985              self._attr_effect_lbl.configure(
986                  text=f"Inspection failed ({attr.error}); attribute restrictions undetermined.",
987              )
988          elif attr.is_readonly is True:
989              if op_str in ("write", "delete"):
990                  self._attr_effect_lbl.configure(
991                      text=f"Read-only attribute is set; prevents {op_str.upper()} operations.",
992                  )
993              else:
994                  self._attr_effect_lbl.configure(
995                      text=f"Read-
only attribute is set, but does not block {op_str.upper()} operations.",
996                  )
997          elif attr.is_readonly is False:
998              self._attr_effect_lbl.configure(
999                  text=f"No attribute restrictions blocking {op_str.upper()} operation.",
1000             )
1001         else:
1002             self._attr_effect_lbl.configure(
1003                 text="Attribute status is unknown or unavailable on this platform.",
1004             )
```

Listing 17. Operation-aware attribute explanation in `MainWindow._update_attributes`: the same read-only flag is described differently for WRITE/DELETE and for other operations.

6. RESULTS AND DISCUSSION

6.1 Evidence capture method

The screenshots in this chapter come from the Review 3 evidence package (version 5, directory `final_screenshots_v3/`). The evidence runner `scripts/capture_review3_evidence.py` creates managed fixtures, drives the real `MainWindow` with the real `DiagnosisEngine`, and captures the genuine window with the Win32 `PrintWindow` call and GDI+. Because the window content is taller than one screen, each scenario is captured at two scroll positions: the top viewport (scroll 0.0: header, input controls, status, process list and permissions) and the bottom viewport (scroll 1.0: status through recommendation, including attributes and cloud sync). No images are stitched, edited or annotated. For compactness this report usually shows the bottom viewport, which contains the status badge and every result panel; the complete top and bottom viewports are shown together for Scenario 03 and are indexed for every scenario in Appendix B.

The process lock in Scenarios 03 and 08 is produced by a controlled helper process that opens the fixture and releases it afterwards; the diagnoser then detects it through Restart Manager. The helper is restricted to the managed demonstration directory.

> **Reading the screenshots**
> Each screenshot is a full-resolution capture reduced to fit the page; the on-screen text is small but legible when the document is zoomed. Scenario IDs (01a … 14) are those of the evidence manifest. The cloud-sync screenshot shows the **negative** boundary case; no live OneDrive or Google Drive lock was demonstrated.

6.2 Demonstration screenshots

Figures 6.1 to 6.16 are genuine captures of the running application. The table lists, for each figure, the scenario, the operation and the diagnosis displayed; all rows except Scenario 05 matched the expected diagnosis.

Table 6.1. Guide to the demonstration figures

Figure	Scenario	Operation	Diagnosis shown	Demonstrates
6.1–6.2	03	WRITE	FILE_LOCKED	Real process lock; complete GUI (top and bottom viewport)
6.3	08	WRITE	FILE_LOCKED	Multiple simultaneous findings
6.4	04a	WRITE	READ_ONLY	Read-only attribute blocks WRITE
6.5	04b	READ	OPERATION_ALLOWED	Read-only contrast with READ
6.6	01b	WRITE	OPERATION_ALLOWED	Normal accessible file
6.7	02	READ	FILE_NOT_FOUND	Missing file; downstream checks skipped
6.8	06	READ	OPERATION_ALLOWED	Hidden and archive attributes
6.9	13	READ	OPERATION_ALLOWED	System and archive attributes
6.10	10	EXECUTE	OPERATION_ALLOWED	EXECUTE diagnostic check
6.11	11	DELETE	OPERATION_ALLOWED	DELETE diagnostic check
6.12	12	DELETE	READ_ONLY	DELETE blocked by read-only attribute
6.13	14	READ	OPERATION_ALLOWED	Stale-result clearing after re-diagnosis
6.14	09	READ	UNKNOWN_ERROR	Genuine inspection error (directory target)
6.15	07	READ	OPERATION_ALLOWED	Cloud-sync boundary (no association detected)
6.16	05	READ	UNKNOWN_ERROR (BLOCKED)	Permission restriction not reproduced

6.2.1 Complete interface: top and bottom viewports of one diagnosis (Scenario 03)

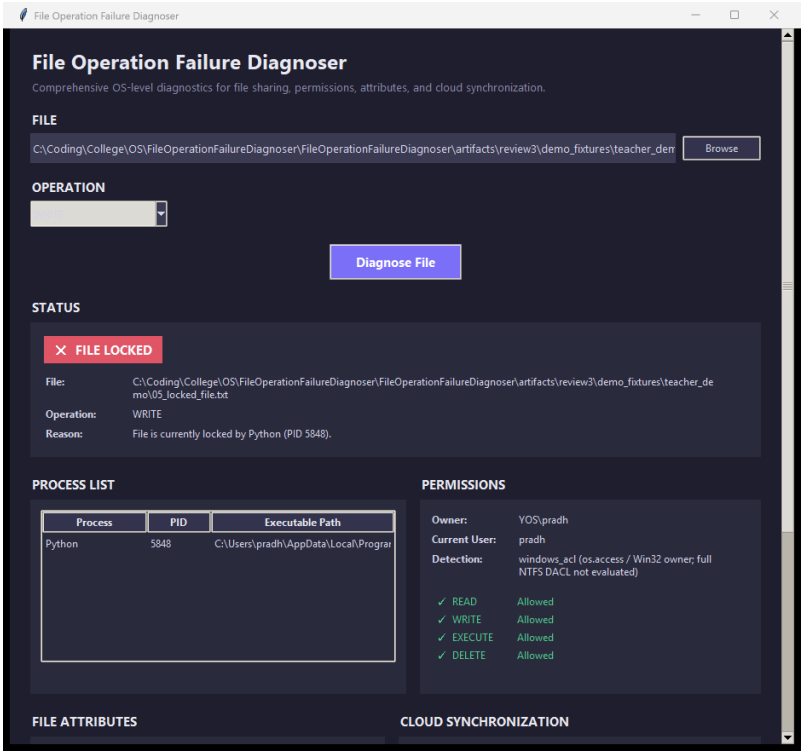


Figure 6.1. Scenario 03, **top viewport**. WRITE on `05_locked_file.txt` while a helper process holds it open. Diagnosis: FILE LOCKED — "File is currently locked by Python (PID 5848)." The operation drop-down text is not visible in the captured image; the selected operation (WRITE) appears in the Status panel.

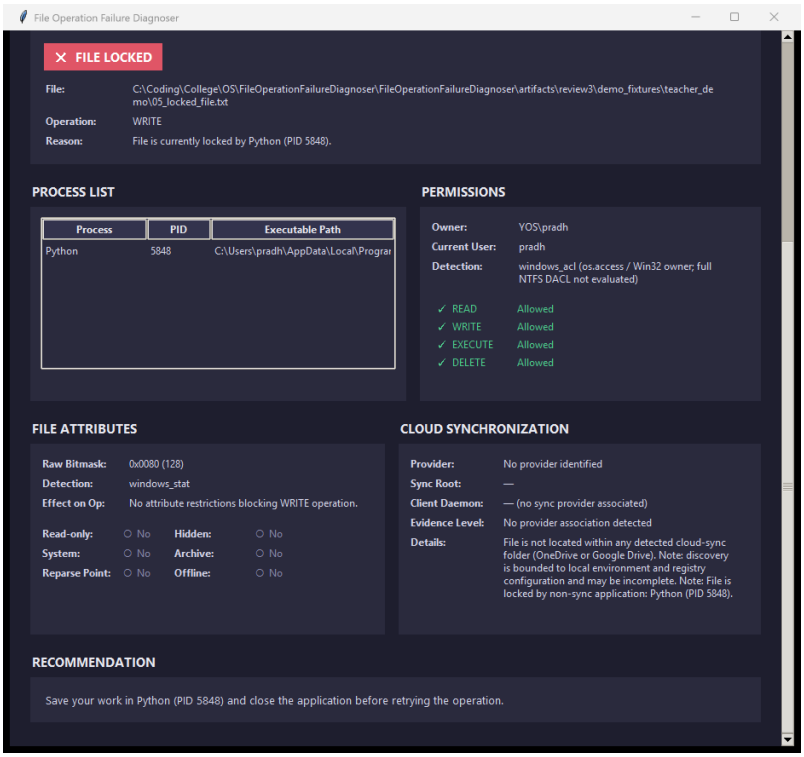


Figure 6.2. Scenario 03, **bottom viewport** of the same diagnosis. Process List: Python, PID 5848, executable path from `psutil`. Permissions: all four operations allowed. Attributes: 0x0080. Cloud Synchronization: no provider identified, with the note that the file is locked by a non-sync application. Recommendation: save work in Python (PID 5848) and close it before retrying.

6.2.2 Multiple simultaneous findings (Scenario 08)

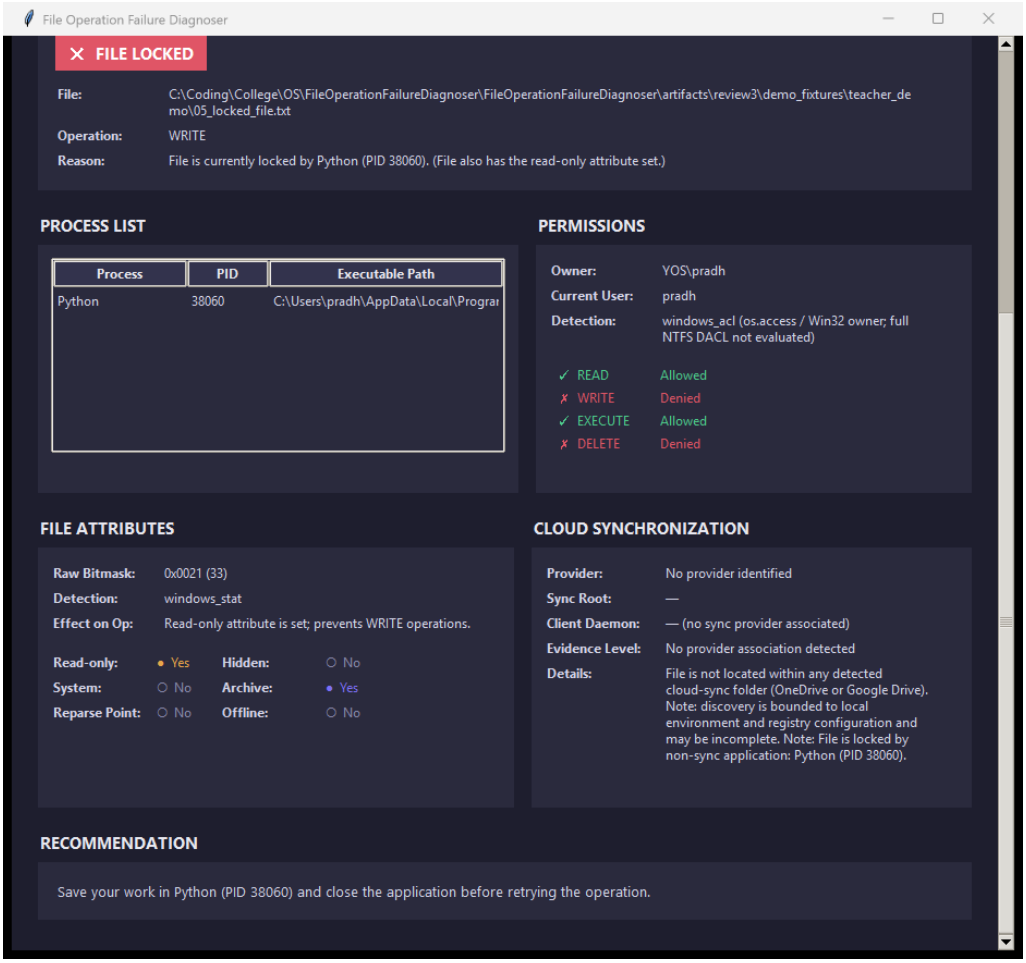


Figure 6.3. Scenario 08: WRITE on a file that is both held open and read-only. The primary diagnosis is FILE LOCKED (Python, PID 38060 — a separate run from Scenario 03). The reason carries the secondary note “File also has the read-only attribute set”; the Permissions panel shows WRITE and DELETE denied, and the Attributes panel shows 0x0021 with Read-only = Yes.

Table 6.2. Evidence observed in Scenario 08 (Figure 6.3)

Evidence	Observed in the screenshot
Primary diagnosis (Rule 2)	FILE LOCKED, with holder name and PID returned by Restart Manager
Secondary evidence preserved	Read-only attribute noted in the reason and shown in the Attributes panel
Advice	Names the holding process first; the read-only attribute remains visible for the next step

6.2.3 Read-only attribute: WRITE versus READ (Scenarios 04a and 04b)

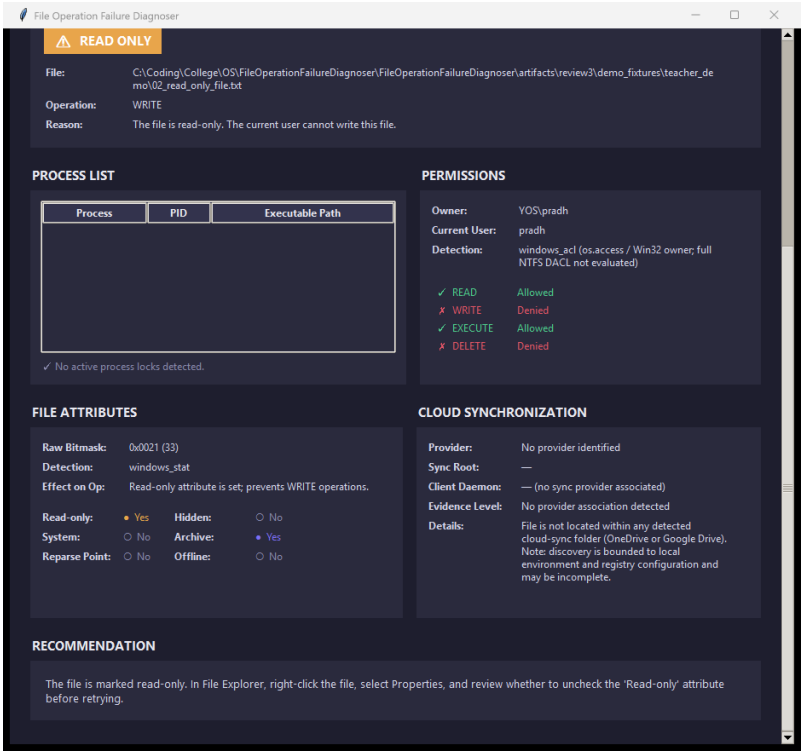


Figure 6.4. Scenario 04a: WRITE on `02_read_only_file.txt` → READ ONLY. Attributes 0x0021 (read-only, archive); effect text “Read-only attribute is set; prevents WRITE operations.” Permissions: WRITE and DELETE denied, READ and EXECUTE allowed. No process lock.

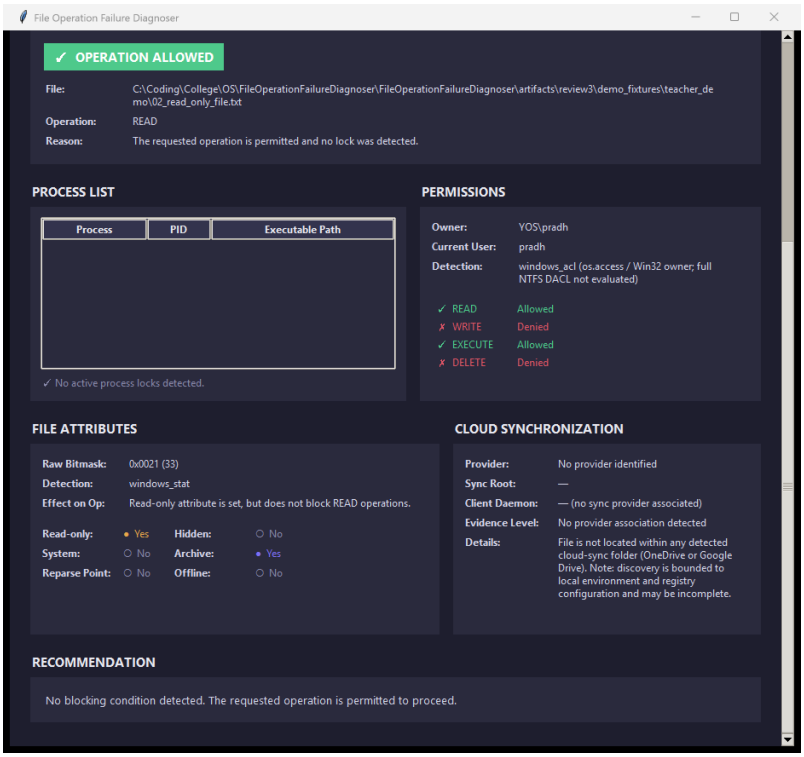


Figure 6.5. Scenario 04b: the same file with READ → OPERATION ALLOWED. The read-only flag is still shown, but the effect text states it does not block READ operations. The operation, not only the file, determines the diagnosis.

6.2.4 Baseline cases: accessible file and missing file (Scenarios 01b and 02)

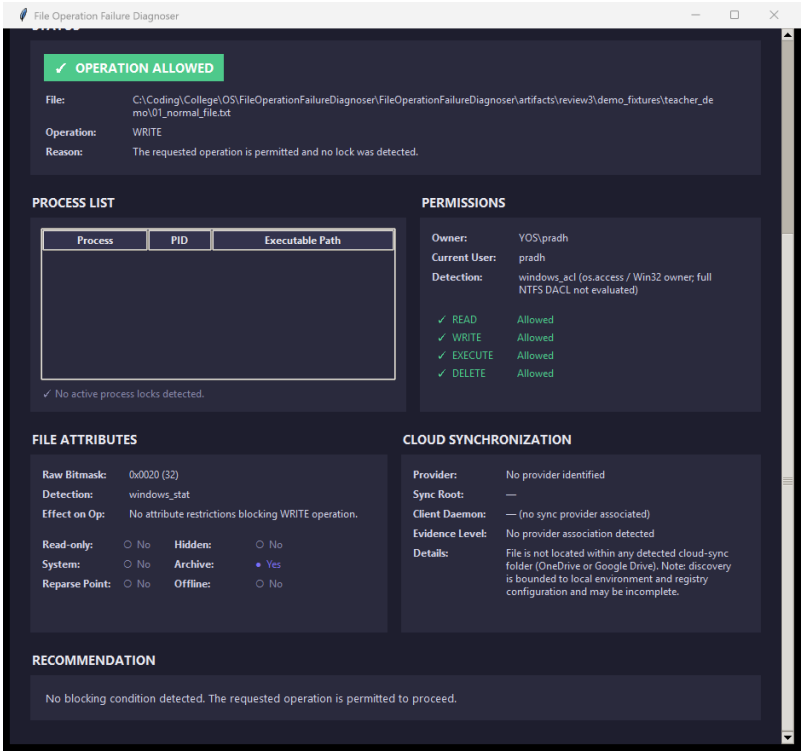


Figure 6.6. Scenario 01b: WRITE on `01_normal_file.txt` → OPERATION ALLOWED. No locks, all four permissions allowed, attributes 0x0020 (archive only), and “No provider association detected”. Recommendation: no blocking condition detected.

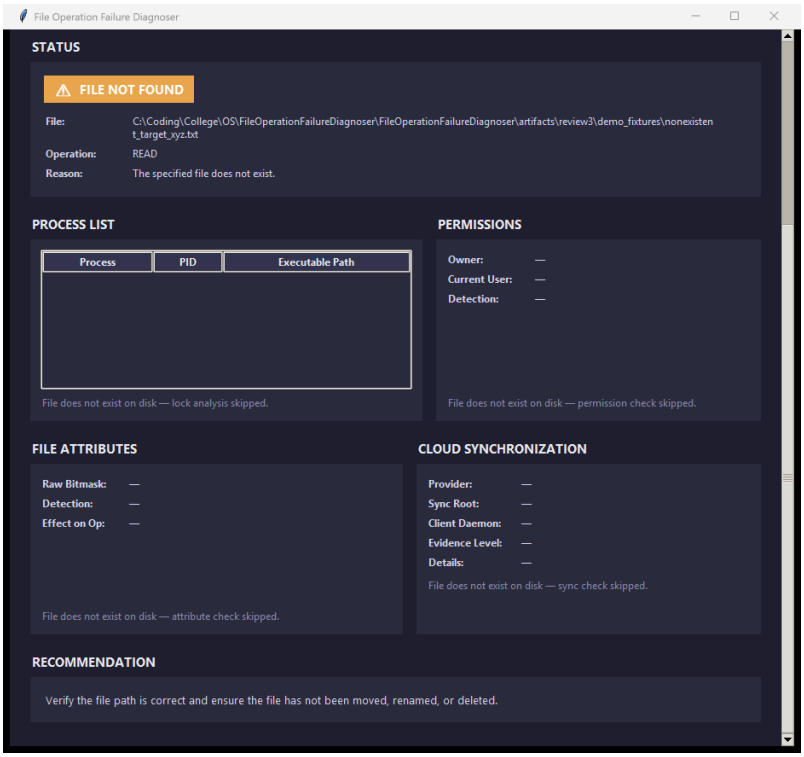


Figure 6.7. Scenario 02: READ on a path that does not exist → FILE NOT FOUND. Each downstream panel states that its check was skipped (“lock analysis skipped”, “permission check skipped”, …) rather than showing misleading empty success.

6.2.5 Attribute evidence: hidden, archive and system flags (Scenarios 06 and 13)

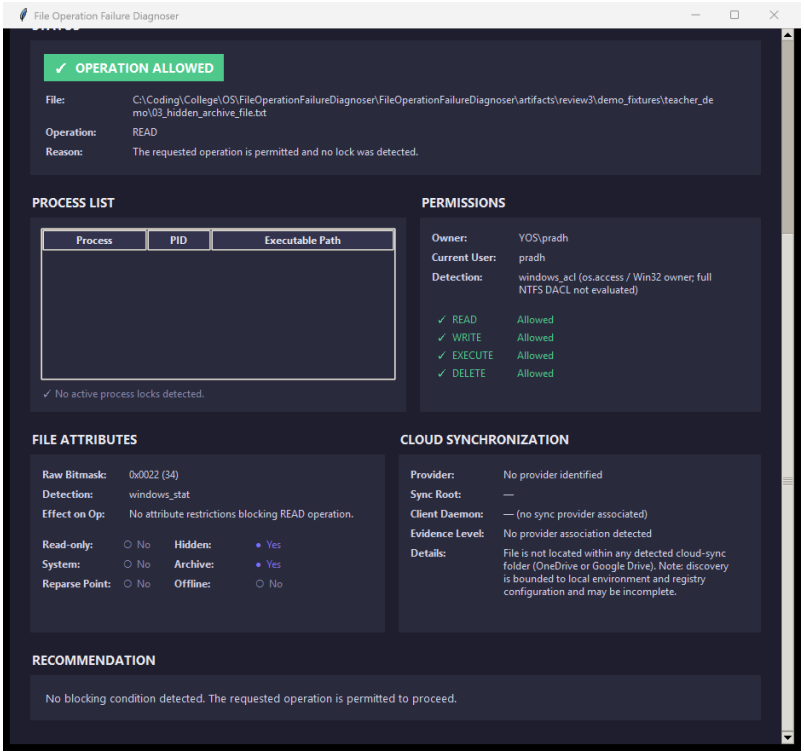


Figure 6.8. Scenario 06: READ on `03_hidden_archive_file.txt` → OPERATION ALLOWED. Raw bitmask 0x0022 decoded as Hidden = Yes and Archive = Yes. The flags are informational and do not block READ.

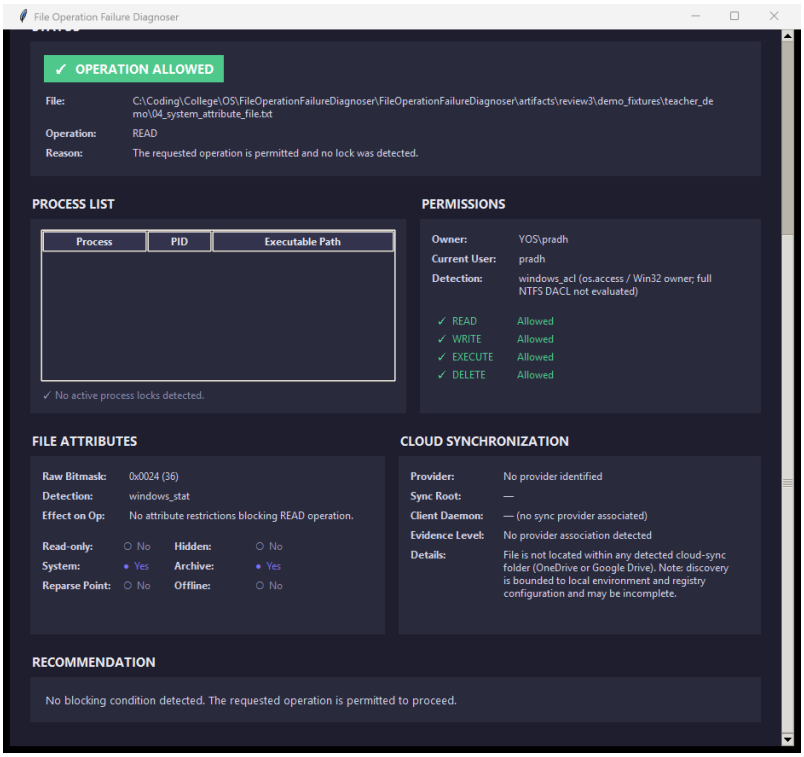


Figure 6.9. Scenario 13: READ on `04_system_attribute_file.txt` → OPERATION ALLOWED. Raw bitmask 0x0024 decoded as System = Yes and Archive = Yes; no attribute restriction applies to READ.

6.2.6 EXECUTE and DELETE diagnostic checks (Scenarios 10 and 11)

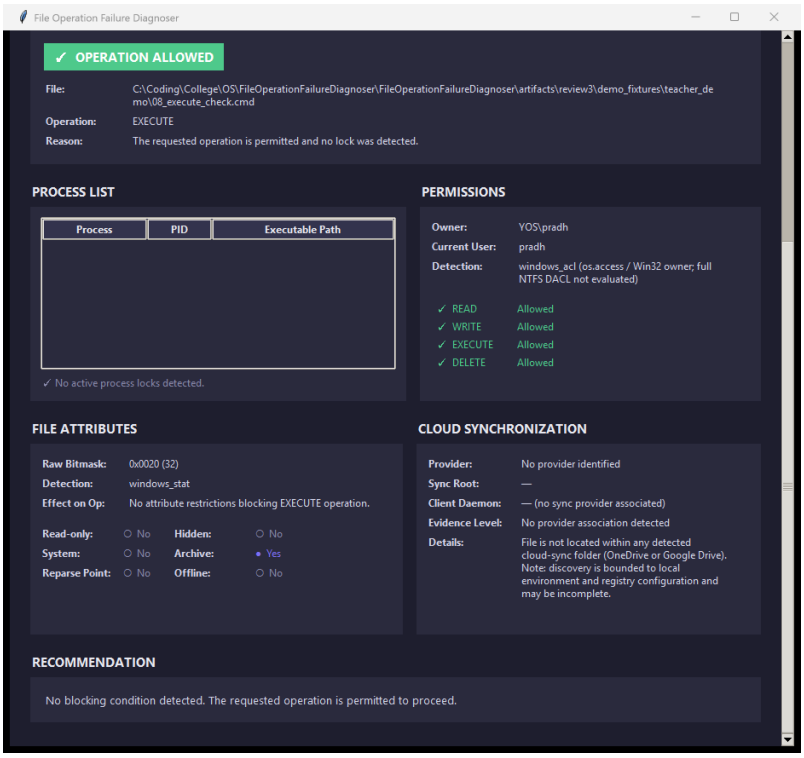


Figure 6.10. Scenario 10: EXECUTE on `08_execute_check.cmd` → OPERATION ALLOWED. The diagnoser evaluates EXECUTE rights statically; the script is never run.

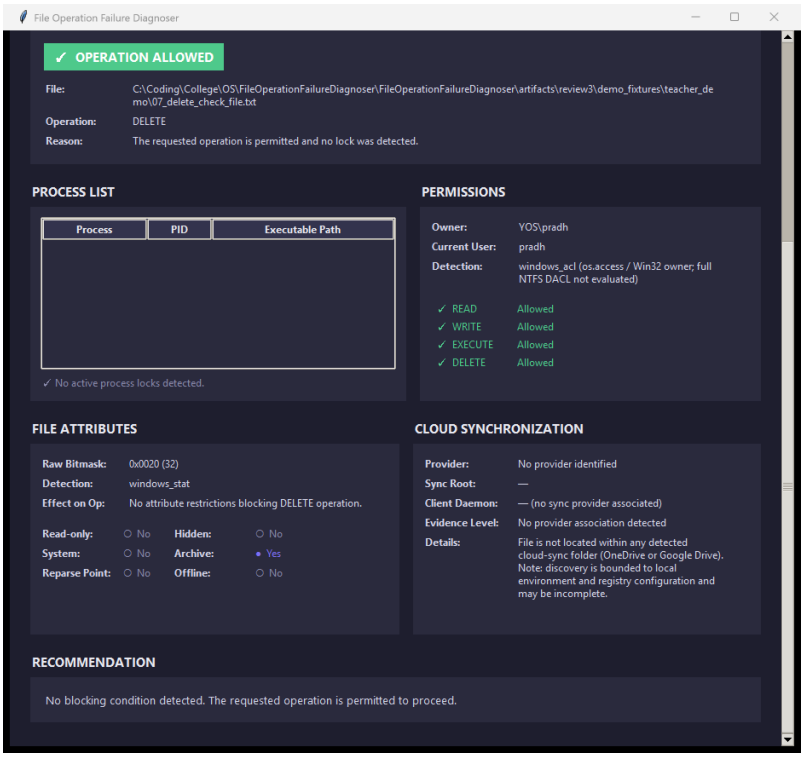


Figure 6.11. Scenario 11: DELETE on `07_delete_check_file.txt` → OPERATION ALLOWED. The file is only inspected; the diagnosis does not delete it.

6.2.7 DELETE on a read-only file and stale-result clearing (Scenarios 12 and 14)

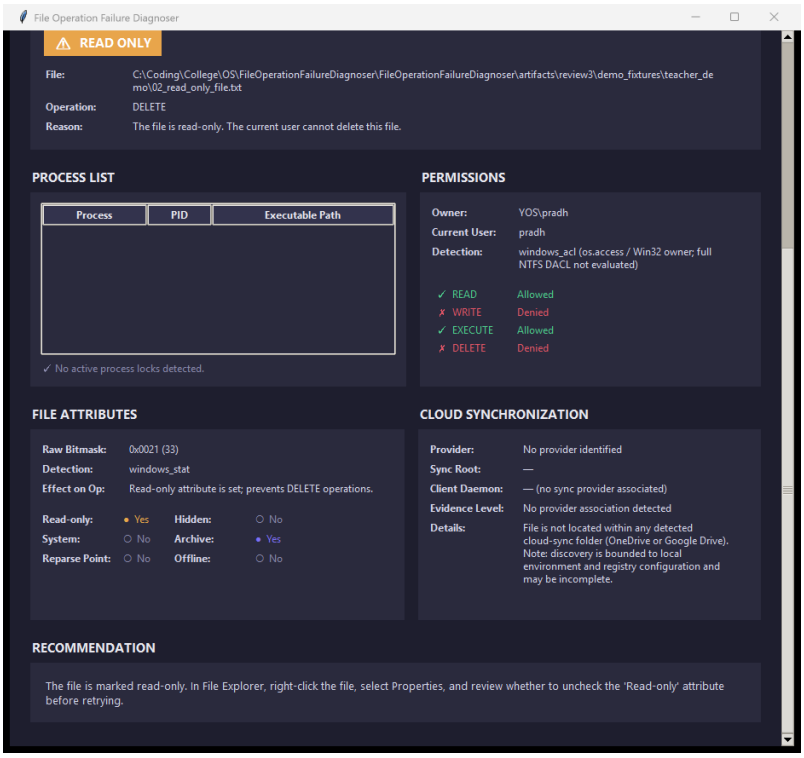


Figure 6.12. Scenario 12: DELETE on the read-only fixture → READ ONLY. Effect text: “Read-only attribute is set; prevents DELETE operations.” Permissions show DELETE denied.

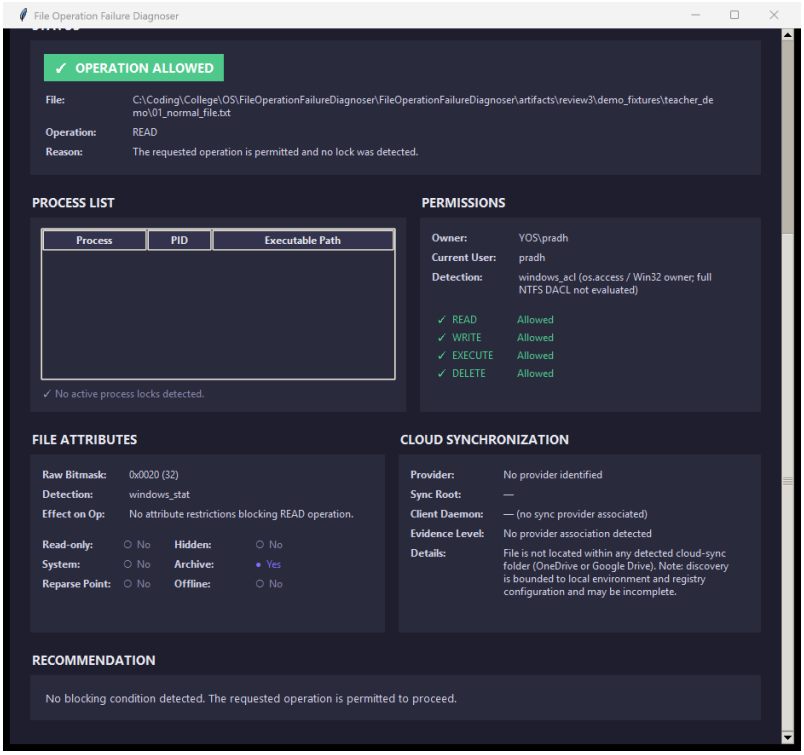


Figure 6.13. Scenario 14: a normal file diagnosed after earlier runs in the same window. The screenshot shows the final cleared state (empty Process List, OPERATION ALLOWED); the run log records Process List count 0 and attributes the clean state to `_reset_panels()`. The earlier locked state is not itself visible in this image.

6.2.8 A genuine inspection error and the cloud-sync boundary (Scenarios 09 and 07)

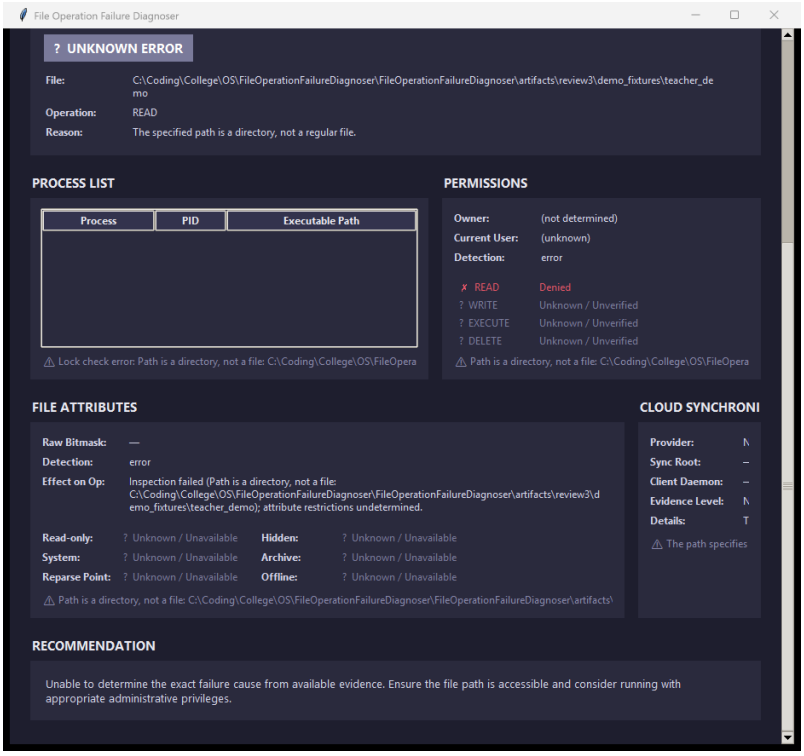


Figure 6.14. Scenario 09: a directory selected as the target → UNKNOWN ERROR, “The specified path is a directory, not a regular file.” Each analyzer returns a structured error and the panels show Unknown / Unverified. Very long error paths are not wrapped and push the right-hand Cloud Synchronization card partly outside the window (Section 6.12).

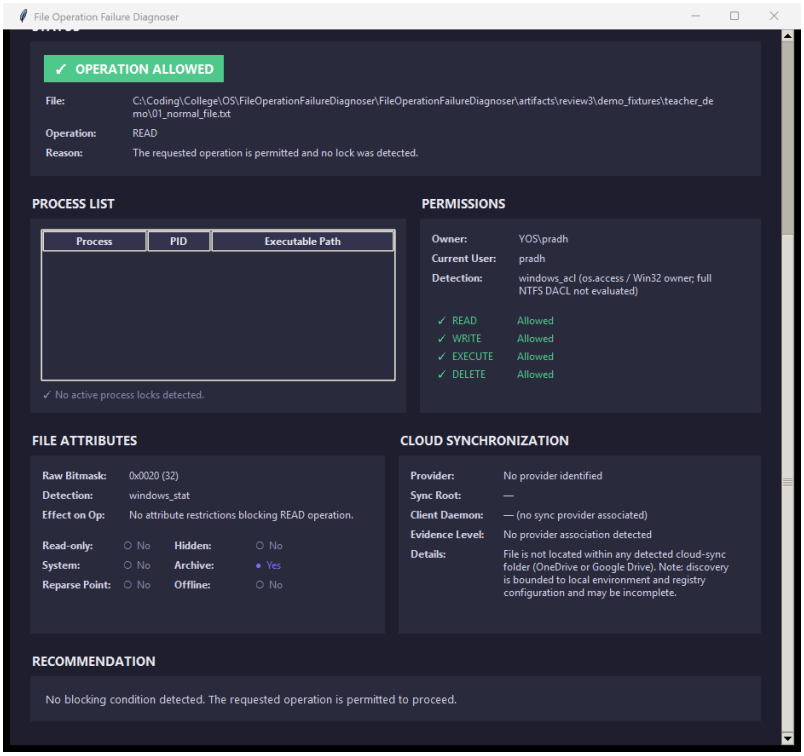


Figure 6.15. Scenario 07: cloud-sync **boundary** behaviour for `01_normal_file.txt`, which lies outside any detected OneDrive or Google Drive folder. The panel reports “No provider association detected” and notes that discovery is bounded and may be incomplete. This is not a sync-conflict demonstration.

6.2.9 Permission-restriction scenario: BLOCKED (Scenario 05)

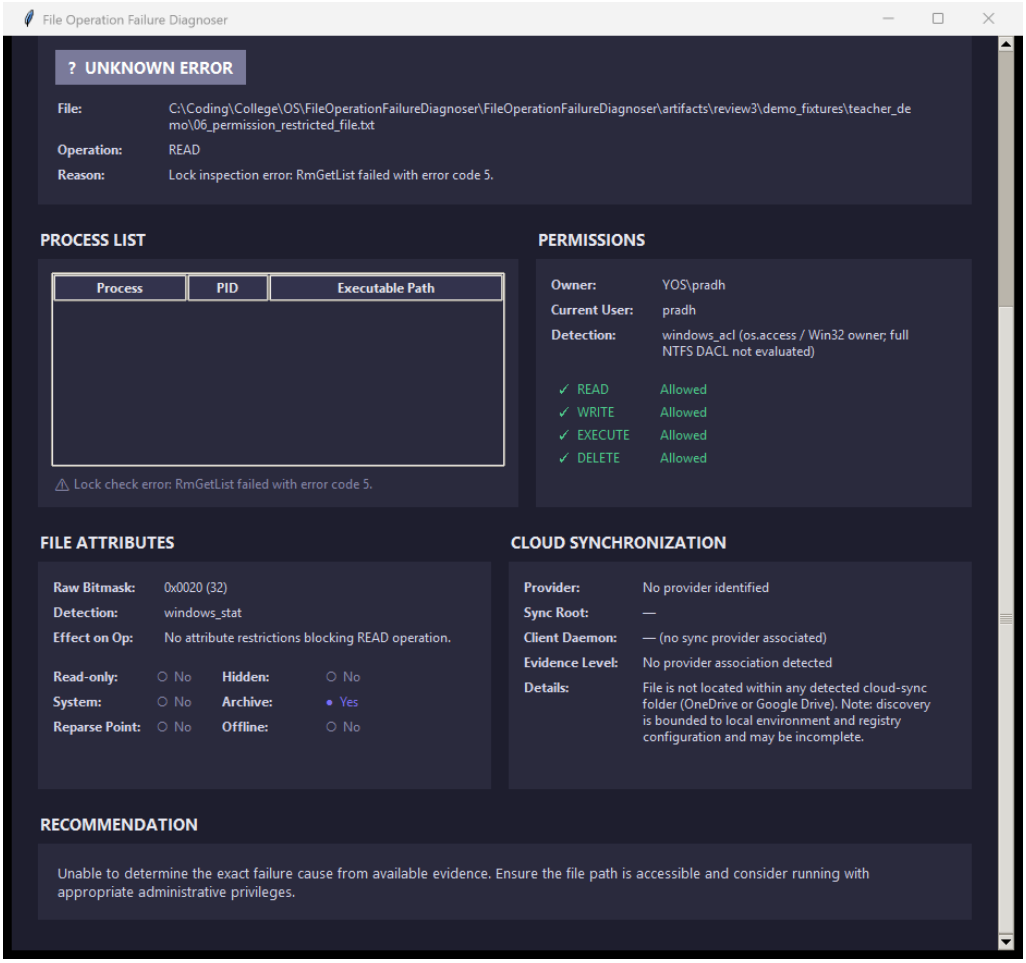


Figure 6.16. Scenario 05 (**BLOCKED**): READ on `06_permission_restricted_file.txt`, which carries an explicit deny ACE. Expected diagnosis PERMISSION_DENIED; actual diagnosis UNKNOWN ERROR, "Lock inspection error: RmGetList failed with error code 5." The Permissions panel still lists READ as allowed because `os.access()` is preliminary evidence and the DACL is not evaluated.

> **This is not a successful PERMISSION_DENIED demonstration**
> The live run did not reproduce the expected diagnosis. `RmGetList` returned Win32 error 5, so the lock check failed and the engine — correctly refusing to guess — reported UNKNOWN_ERROR. The report does not assert why Windows returned that code for this file. The PERMISSION_DENIED rules are covered by automated tests with injected evidence, not by a live capture, and the scenario is recorded as BLOCKED in the evidence manifest.

6.3 Testing methodology

Verification combines automated tests with live demonstrations. The two are reported separately because they establish different things: automated tests with injected evidence prove that decision logic behaves as designed for conditions that are hard or unsafe to create on a real machine; live demonstrations prove that the whole application works on real Windows state.

Table 6.3. Verification layers

Layer	What it exercises	Approach
Model and recommender tests	Dataclass defaults, enums, recommendation text per diagnosis	Pure unit tests
Analyzer tests (cross-platform classes)	Path handling, structured errors, result structure, sync root matching, evidence tiers	Temporary files, patched roots and processes, injected overrides
Analyzer tests (Windows classes)	Real Restart Manager, real attributes, real host discovery	`skipif(sys.platform != "win32")`; run against real files
Engine tests	Priority policy, secondary notes, exception isolation, sync-lock correlation, integration across all four peers	Injected analyzer results plus integration tests
GUI tests	Panel population, reset behaviour, error badges, DELETE row, sync labels	Real Tk root hidden from view; mocked engine and constructed `DiagnosisResult` objects
Live evidence run	Whole application on real fixtures: locks, attributes, operations, directory, blocked ACL case	Real GUI and engine; native window capture; assertion of the returned diagnosis

6.4 Final regression result

Table 6.4. Final verified regression record

Item	Value
Tests collected	157
Passed	156
Failed	0
Skipped	1
Environment	Windows 11 Pro 64-bit · Python 3.13.14 · pytest 9.1.1
Reported run time	4.20 s

Table 6.5. Final-run test counts by file (from the regression log)

Test file	Tests	Passed	Skipped
`tests/test_diagnoser.py`	43	43	0
`tests/test_sync_conflict.py`	20	20	0
`tests/test_permissions.py`	19	19	0
`tests/test_models.py`	23	23	0
`tests/test_gui.py`	17	17	0
`tests/test_recommender.py`	15	15	0
`tests/test_file_attributes.py`	12	12	0
`tests/test_process_lock.py`	8	7	1
Total	157	156	1

6.5 Historical milestone totals (kept separate)

The engineering log recorded a regression total at each milestone. These are historical snapshots taken at different times as the suite grew; they are shown for traceability only and are not added to, or merged with, the final result above.

Table 6.6. Historical totals as recorded at each milestone

Milestone	Collected	Passed	Failed	Skipped	Environment as recorded
Review 2 baseline	72	71	0	1	Windows (Review 2 report)
Sprint 1	104	103	0	1	Windows (log)
Sprint 2	124	123	0	1	Windows (log)
Sprint 3	139	138	0	1	Windows, Python 3.13.14 (log)
Sprint 4	155	154	0	1	Windows (log)
Final project audit	156	155	0	1	Windows 11 Pro 64-bit, Python 3.13.14, pytest 9.1.1 (log)
Final verified record	157	156	0	1	Windows 11 Pro 64-bit, Python 3.13.14, pytest 9.1.1

6.6 The skipped test

The single skipped test is `tests/test_process_lock.py::TestProcessLockAnalyzerCrossPlatform::test_unsupported_platform_message`. It verifies the message that `ProcessLockAnalyzer` returns on a non-Windows platform. It is decorated with `skipif(sys.platform == "win32")`, so on the Windows test machine it is skipped by design (its reason string is “Non-Windows platform check only”). Its Windows counterparts run and pass. The skip is therefore a platform-specific condition, not a failure and not an untested Windows behaviour.

6.7 Live demonstration counts

Table 6.7. Live GUI demonstration counts

Measure	Result
Logical scenario groups	14
Individual scenario executions	16 (Scenarios 01 and 04 each run for two operations: 01a/01b, 04a/04b)
Passed against expected diagnosis	15 executions across 13 logical groups
Blocked	1 execution (Scenario 05, permission restriction)
Run recorded	2026-10-18, Windows 11 Pro 64-bit, Tkinter 8.6

6.8 Scenario matrix

Table 6.8. Live GUI scenario results (Review 3 evidence manifest, v5)

ID	Scenario	Op.	Expected	Actual	Status
01a	Normal accessible file	READ	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
01b	Normal accessible file	WRITE	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
02	Nonexistent file	READ	`FILE_NOT_FOUND`	`FILE_NOT_FOUND`	PASS
03	Active process lock	WRITE	`FILE_LOCKED`	`FILE_LOCKED`	PASS
04a	Read-only attribute	WRITE	`READ_ONLY`	`READ_ONLY`	PASS
04b	Read-only attribute (contrast)	READ	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
05	NTFS permission restriction (deny ACE)	READ	`PERMISSION_DENIED`	`UNKNOWN_ERROR`	BLOCKED
06	Hidden + archive attributes	READ	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
07	Cloud-sync boundary (negative)	READ	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
08	Lock + read-only (multiple findings)	WRITE	`FILE_LOCKED`	`FILE_LOCKED`	PASS
09	Directory as target	READ	`UNKNOWN_ERROR`	`UNKNOWN_ERROR`	PASS
10	EXECUTE check	EXECUTE	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
11	DELETE check, allowed	DELETE	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
12	DELETE on read-only file	DELETE	`READ_ONLY`	`READ_ONLY`	PASS
13	System attribute	READ	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS
14	Sequential re-diagnosis, no stale data	READ	`OPERATION_ALLOWED`	`OPERATION_ALLOWED`	PASS

6.9 What each kind of evidence establishes

Table 6.9. Live demonstration versus automated evidence

Behaviour	Live GUI demonstration	Automated tests
Process lock with returned PID, name and path	Yes — Scenarios 03 and 08	Yes (injected and live Restart Manager tests)
Read-only blocks WRITE and DELETE but not READ	Yes — 04a, 04b, 12	Yes
Hidden, system, archive decoding	Yes — 01, 06, 13	Yes
EXECUTE and DELETE diagnostic checks	Yes — 10, 11, 12	Yes
Multiple findings and secondary notes	Yes — 08	Yes
Panel reset between diagnoses	Yes — 14 (final cleared state)	Yes (`test_gui.py`)
Cloud-sync negative boundary	Yes — 07 (“No provider association detected”)	Yes
Confirmed OneDrive / Google Drive lock → `SYNC_CONFLICT`	Not demonstrated live	Yes — injected lock and sync-root evidence
Sync-folder path or running client is not a conflict	Not demonstrated live	Yes
`PERMISSION_DENIED` classification	BLOCKED — not reproduced (Scenario 05)	Yes — injected permission evidence

The evidence manifest defines a NOT_REPRODUCED status for cases where safety prevents live creation of test files inside real OneDrive or Google Drive folders. Accordingly, no positive live sync-lock scenario is claimed in this report.

6.10 Key findings

1. **Operation semantics matter.** The same read-only file is READ ONLY for WRITE and DELETE but OPERATION ALLOWED for READ (Scenarios 04a, 04b, 12). A tool that reports file properties without the intended operation would mislead.
2. **Priority and secondary evidence work together.** With a lock and a read-only attribute present (Scenario 08), the engine names the lock as primary and still surfaces the read-only attribute, so the user can anticipate the next obstacle.
3. **Declining to guess is a feature.** In Scenario 05 the engine reported UNKNOWN_ERROR rather than fabricating PERMISSION_DENIED when the Restart Manager query failed with Win32 error 5. Reporting an honest unknown is consistent with the conservative-claims principle.
4. **Restart Manager is useful evidence for live holders.** It returned the holding application name and PID, and the path through `psutil`, in the live captures (Scenarios 03 and 08) without terminating or modifying anything; the report does not claim it enumerates every open handle.
5. **Cloud-sync evidence must be tiered.** Association with a sync root and a running client are common and benign; only a holder matching a provider client, correlated with the sync root, justifies a conflict. The implementation encodes this and the tests guard it.
6. **Evidence quality is split clearly.** Live demonstrations and automated tests cover different behaviours (Section 6.9), and the report does not blur them.

6.11 Limitations

Table 6.10. Documented limitations of the final implementation

Area	Limitation	Consequence
Permission evaluation	`os.access()` and owner lookup are used; the implementation requests owner information only, so the DACL (including inheritance, group membership and explicit deny ACEs) is not retrieved or evaluated.	The Permissions panel is preliminary evidence, not a prediction: it can show “Allowed” for an operation that Windows will deny (Scenario 05 lists READ as allowed).
DELETE semantics	DELETE is approximated by write access; parent-directory delete rights are not examined.	A file may be reported deletable although the containing folder forbids it.

Area	Limitation	Consequence
Restart Manager scope	Restart Manager was designed for installers and reports the applications and services using the registered resource; it is not documented as a complete enumeration of all open handles, and the query can itself fail (Win32 error 5 was observed).	An empty process list is not proof that nothing holds the file; failures become UNKNOWN_ERROR.
Point-in-time snapshot	Evidence is collected once per diagnosis; processes may start or exit afterwards, and a path to an executable may not be resolvable.	Results can change on retry; the path column may be blank.
Cloud-sync coverage	Only Microsoft OneDrive and Google Drive; discovery uses environment variables, registry values and local drive information; custom or unusual setups may not be found.	A file in an undiscovered sync folder is reported as having no provider association.
Live sync validation	No positive live OneDrive / Google Drive lock was demonstrated; confirmed-lock logic is verified with injected evidence.	Real-world provider behaviour is not empirically validated.
Platform	Primary mechanisms are Windows-specific; other platforms receive fallbacks or an unsupported-platform message.	The tool is intended for Windows 10/11 only.
Test environment	Final results come from a single Windows 11 machine and user account with local NTFS fixtures.	Network shares, other file systems and multi-user scenarios were not tested.

6.12 Observed GUI limitations

Two behaviours were visible in the captured screenshots and are reported here as observed rather than investigated further. First, in the Scenario 09 screenshot a very long error path is not wrapped, which widens the content and pushes the Cloud Synchronization card partly outside the visible window. Second, in the top-viewport captures the operation drop-down appears without visible text although the selected operation is correctly shown in the Status panel; this may be a styling or window-capture artefact.

6.13 Discussion

The project connects several core operating-system topics to a practical tool. Process management appears in the identification of holders by PID. System interfaces appear in the use of Restart Manager and Win32 security calls through `ctypes`. File systems and protection appear in attributes, ownership and the difference between access flags and a full access-control list.

Compared with the tools surveyed in Chapter 2, the Diagnoser trades breadth of action for clarity of explanation: it never unlocks or kills anything, and its stated boundaries — advisory only, no DACL claims, two providers — make its output trustworthy.

7. CONCLUSION

7.1 Conclusion

The File Operation Failure Diagnoser meets the objectives set in Review 1 within the safe, advisory scope adopted in Review 2. The final system is a Tkinter application whose Diagnosis Engine coordinates four peer analyzers — Process / Lock, Permission / Ownership, File Attribute and Cloud-Sync — correlates their structured evidence with a deterministic priority policy, and presents one primary diagnosis with preserved secondary findings and an evidence-grounded recommendation.

Verification is reported at two levels. The final regression record shows 157 tests collected, 156 passed, 0 failed and 1 skipped on Windows 11 Pro 64-bit with Python 3.13.14 and pytest 9.1.1; the skipped test is the platform-specific non-Windows check. The live GUI suite covered fourteen logical groups in sixteen executions, of which fifteen matched expectation and one — the explicit-deny-ACE permission case — was BLOCKED because the Restart Manager query failed with Win32 error 5, and was reported honestly as UNKNOWN_ERROR. The cloud-sync demonstration shows the negative boundary only; confirmed provider-lock classification is established by automated tests.

7.2 Future improvements

The following are directions suggested by the documented limitations. None is implemented in the delivered system.

Table 7.1. Future work (not implemented)

Direction	Motivation from this report
Stronger Windows permission evaluation: retrieve the DACL and compute effective rights for the current token, including inheritance and explicit deny entries	Scenario 05 and Section 6.11: deny ACEs are currently invisible and DELETE is approximated
Parent-directory rights for DELETE	DELETE currently relies on file write access
More comprehensive provider integration and discovery (additional providers and non-default sync locations)	Only OneDrive and Google Drive are supported; discovery is bounded
Expanded live sync validation with dedicated test accounts and folders, so that a confirmed sync-lock scenario can be captured safely	No positive live sync-lock scenario exists; the manifest records such cases as not reproduced for safety
GUI refinements: wrap long error paths and verify the operation selector rendering	Section 6.12 observations

REFERENCES

IEEE format; numbered in order of first citation. “Accessed” gives the date a page was opened and read for this report; Microsoft Learn pages are the official documentation for the Windows interfaces named.

[1] T. Anderson and M. Dahlin, *Operating Systems: Principles and Practice*, 2nd ed. Recursive Books, 2014.

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[3] M. E. Russinovich, D. A. Solomon, and A. Ionescu, *Windows Internals, Part 1*, 6th ed. Redmond, WA, USA: Microsoft Press, 2012.

[4] Microsoft Corporation, “Sync your computer’s files and folders with OneDrive,” Microsoft Support. [Online]. Available: https://support.microsoft.com/en-us/onedrive/sync-your-computer-s-files-and-folders-with-onedrive Accessed: Oct. 19, 2026.

[5] Google LLC, “Stream & mirror files with Drive for desktop,” Google Drive Help. [Online]. Available: https://support.google.com/drive/answer/13401938?hl=en Accessed: Oct. 19, 2026.

[6] M. Russinovich, “Handle v5.0,” Sysinternals, Microsoft Learn, Oct. 26, 2022. [Online]. Available: https://learn.microsoft.com/en-us/sysinternals/downloads/handle Accessed: Oct. 19, 2026.

[7] Microsoft Corporation, “File Locksmith utility,” PowerToys, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/powertoys/file-locksmith Accessed: Oct. 19, 2026.

[8] Crystal Rich Ltd., “LockHunter is a free 64/32 bit tool to delete files blocked by any processes,” LockHunter. [Online]. Available: https://lockhunter.com/ Accessed: Oct. 19, 2026.

[9] Microsoft Corporation, “RmStartSession function (restartmanager.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/restartmanager/nf-restartmanager-rmstartsession Accessed: Oct. 19, 2026.

[10] Microsoft Corporation, “RmRegisterResources function (restartmanager.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/restartmanager/nf-restartmanager-rmregisterresources Accessed: Oct. 19, 2026.

[11] Microsoft Corporation, “RmGetList function (restartmanager.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/restartmanager/nf-restartmanager-rmgetlist Accessed: Oct. 19, 2026.

[12] Microsoft Corporation, “RmEndSession function (restartmanager.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/restartmanager/nf-restartmanager-rmendsession Accessed: Oct. 19, 2026.

[13] Microsoft Corporation, “GetNamedSecurityInfoW function (aclapi.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/aclapi/nf-aclapi-getnamedsecurityinfow Accessed: Oct. 19, 2026.

[14] Microsoft Corporation, “LookupAccountSidW function (winbase.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-lookupaccountsidw Accessed: Oct. 19, 2026.

[15] Python Software Foundation, “os.access(),” in “os — Miscellaneous operating system interfaces,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/os.html#os.access Accessed: Oct. 19, 2026.

[16] Python Software Foundation, “os.stat_result,” in “os — Miscellaneous operating system interfaces,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/os.html#os.stat_result Accessed: Oct. 21, 2026.

[17] Microsoft Corporation, “GetFileAttributesW function (fileapi.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-getfileattributesw Accessed: Oct. 19, 2026.

[18] Microsoft Corporation, “GetVolumeInformationW function (fileapi.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-getvolumeinformationw Accessed: Oct. 21, 2026.

[19] Python Software Foundation, “winreg — Windows registry access,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/winreg.html Accessed: Oct. 21, 2026.

[20] G. Rodola, “psutil documentation,” Read the Docs. [Online]. Available: https://psutil.readthedocs.io/stable/ Accessed: Oct. 21, 2026.

[21] Python Software Foundation, “ctypes — A foreign function library for Python,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/ctypes.html Accessed: Oct. 21, 2026.

[22] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/tkinter.html Accessed: Oct. 21, 2026.

[23] pytest-dev, “pytest documentation.” [Online]. Available: https://docs.pytest.org/en/stable/ Accessed: Oct. 19, 2026.

[24] Microsoft Corporation, “About Restart Manager,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/rstmgr/about-restart-manager Accessed: Oct. 19, 2026.

[25] Microsoft Corporation, “Restart Manager functions,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/rstmgr/functions Accessed: Oct. 19, 2026.

[26] Microsoft Corporation, “How DACLs Control Access to an Object,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/secauthz/how-dacls-control-access-to-an-object Accessed: Oct. 19, 2026.

[27] Microsoft Corporation, “File attribute constants,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/fileio/file-attribute-constants Accessed: Oct. 19, 2026.

APPENDIX A. ADDITIONAL SOURCE EXCERPTS

The excerpts below are copied verbatim from the supplied source; line numbers refer to the files named in each header. They supplement the listings in Chapters 4 and 5.

A.1 Failure isolation in the Diagnosis Engine

`_safe_analyze_*` wrappers convert an unexpected analyzer exception into a structured error result; the behaviour is covered by `test_analyzer_exception_isolated_and_converted_to_error_result` in `tests/test_diagnoser.py`.

src/diagnoser/engine/diagnosis.py lines 212–221

```
212      def _safe_analyze_lock(self, path: str) -> LockDetectionResult:
213          try:
214              return self._lock_analyzer.analyze(path)
215          except Exception as exc:
216              return LockDetectionResult(
217                  file_path=path,
218                  is_locked=False,
219                  error=f"Lock analysis error: {exc}",
220                  detection_method="error",
221              )
```

Listing 18. Exception isolation for the Process / Lock Analyzer call in `DiagnosisEngine`.

A.2 Analyzer base class

src/diagnoser/analyzers/base.py lines 13–42

```
13  class BaseAnalyzer(ABC):
14      """Abstract base for all file-operation-failure analyzers.
15
16      Subclasses must implement the analyze() method. This base class
17      provides common helper methods for input validation.
18      """
19
20      @abstractmethod
21      def analyze(self, file_path: str):
22          """Analyse the given file path and return a structured result.
23
24          Args:
25              file_path: Absolute path to the file to analyse.
26
27          Returns:
28              A dataclass result specific to the analyzer type.
29          """
30          raise NotImplementedError
31
32      @staticmethod
33      def validate_path(file_path: str) -> str:
34          """Normalise and return the absolute path.
35
36          Args:
37              file_path: A file path string.
38
39          Returns:
40              The normalised absolute path.
41          """
42          return os.path.abspath(os.path.normpath(file_path))
```

Listing 19. Excerpt of `BaseAnalyzer`, the shared abstract base of all four peer analyzers.

A.3 Sync-root membership and client detection

src/diagnoser/analyzers/sync_conflict.py lines 99–115

```
    try:
        norm_child = os.path.abspath(os.path.normpath(child_path)).rstrip("\\/")
        norm_parent = os.path.abspath(os.path.normpath(parent_path)).rstrip("\\/")

        c_drive, c_rest = os.path.splitdrive(norm_child)
        p_drive, p_rest = os.path.splitdrive(norm_parent)

        if c_drive.lower() != p_drive.lower():
            return False

        c_parts = [p.lower() for p in c_rest.replace("/", "\\").split("\\") if p]
        p_parts = [p.lower() for p in p_rest.replace("/", "\\").split("\\") if p]

        if len(c_parts) < len(p_parts):
            return False

        return c_parts[:len(p_parts)] == p_parts
```

Listing 20. Directory-boundary-safe, case-insensitive path containment test used to decide whether a file is inside a sync root.

src/diagnoser/analyzers/sync_conflict.py lines 540–560

```
        found: List[ProcessInfo] = []
        try:
            for proc in psutil.process_iter(["pid", "name", "exe"]):
                try:
                    p_name = (proc.info.get("name") or "").lower()
                    if p_name in target_names:
                        found.append(
                            ProcessInfo(
                                process_name=proc.info.get("name") or p_name,
                                pid=proc.info.get("pid") or proc.pid,
                                executable_path=proc.info.get("exe") or "",
                            )
                        )
                except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
                    continue
                except Exception:
                    continue
        except Exception:
            pass

        return len(found) > 0, found
```

Listing 21. Local sync-client discovery with `psutil.process_iter` (`SyncConflictAnalyzer`). A running client is evidence of possible contention only.

A.4 Recommendation for a confirmed sync conflict

src/diagnoser/recommendations/recommender.py lines 171–190

```
        # If a lock was confirmed by lock analysis or sync result
        if (sync_res and sync_res.is_conflict) or (lock_res and lock_res.is_locked):
            return (
                f"Wait a few moments for {provider} to finish synchronizing, "
                f"or check the {provider} status in the system tray. If necessary, "
                "temporarily pause syncing before modifying the file."
            )

        # Mere association without confirmed lock
        if sync_res and sync_res.is_synced:
            return (
                f"The file is stored inside a {provider} folder, but no active sync "
                "lock was confirmed. Check if the sync client is currently uploading "
                "or downloading changes."
            )

        return (
            "Wait for background cloud synchronization to complete, or check your "
            "sync client status before retrying."
        )
```

Listing 22. Provider-aware advice in `Recommender`, including the wording for unconfirmed association.

A.5 GUI details

src/diagnoser/gui/main_window.py lines 74–84

```
SYNC_EVIDENCE_LABELS = {
    "none": "No provider association detected",
    "path_only": "Sync-folder association detected",
    "potential_contention": "Potential contention; active lock not confirmed",
    "confirmed_lock": "Confirmed sync-client lock",
}

SYNC_PROVIDER_NAMES = {
    "onedrive": "Microsoft OneDrive",
    "google_drive": "Google Drive",
}
```

Listing 23. Human-readable labels for the cloud-sync evidence levels in `main_window.py`.

src/diagnoser/gui/main_window.py lines 888–891

```
888          method_text = perm.detection_method or "—"
889          if "windows_acl" in method_text or "os_access" in method_text:
890              method_text += " (os.access / Win32 owner; full NTFS DACL not evaluated)"
891          self._perm_method_lbl.configure(text=method_text)
```

Listing 24. The Permissions panel states the DACL boundary beside the detection method.

src/diagnoser/gui/main_window.py lines 677–694

```
677      def _reset_panels(self):
678          """Reset all result panels to their initial neutral state."""
679          # Status panel
680          self._status_badge.configure(
681              text="●  WAITING FOR INPUT",
682              background=_COLOURS["grey"],
683          )
684          self._status_file_lbl.configure(text="—")
685          self._status_op_lbl.configure(text="—")
686          self._status_reason_lbl.configure(
687              text="Select a file and operation, then click Diagnose File.",
688          )
689
690          # Process list
691          for item in self._proc_tree.get_children():
692              self._proc_tree.delete(item)
693          self._proc_placeholder.configure(text="No diagnosis performed yet.")
694
```

Listing 25. `MainWindow._reset_panels` excerpt showing the initial reset of the Status and Process List panels; the remaining method resets Permissions, File Attributes, Cloud Sync and Recommendation panels.

src/diagnoser/gui/app.py lines 13–23

```
13  from diagnoser.gui.main_window import MainWindow
14
15
16  def main():
17      """Create and run the main application window."""
18      app = MainWindow()
19      app.run()
20
21
22  if __name__ == "__main__":
23      main()
```

Listing 26. Application entry point (`gui/app.py`).

APPENDIX B. EVIDENCE INDEX

The evidence package `Review3_Evidence_Pack_v5.zip` contains the screenshot manifest, the evidence coverage matrix, the regression log (`logs/test_results.txt`), the scenario log (`logs/scenario_run.log`), the engineering log (`docs/REVIEW3_ENGINEERING_LOG.md`), a teacher demonstration guide and the paired screenshots in `final_screenshots_v3/`. Each scenario has a top and a bottom viewport PNG.

Table B.1. Screenshot files by scenario

Scenario file stem	Viewport files	Status	Used in this report
`01a_normal_file_read`	`_top.png`, `_bottom.png`	PASS	Matrix only
`01b_normal_file_write`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`02_file_not_found`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`03_process_locked_write`	`_top.png`, `_bottom.png`	PASS	Figures (top + bottom)
`04a_read_only_write`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`04b_read_only_read`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`05_permission_ restricted_blocked`	`_top.png`, `_bottom.png`	BLOCKED	Figure (bottom)
`06_file_attributes_ hidden_archive`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`07_cloud_sync_negative`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`08_multiple_findings_ lock_and_readonly`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`09_inspection_ error_directory`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`10_operation_execute`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`11_operation_delete_allowed`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`12_operation_ delete_readonly`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`13_system_ attribute_evidence`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)
`14_reset_stale_clearing`	`_top.png`, `_bottom.png`	PASS	Figure (bottom)